

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ДОПУЩЕН К ЗАЩИТЕ

Заведующий кафедрой

_____ / Пухова Е. А. /

Руководитель образовательной программы

_____ / Даньшина М. В. /

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

по теме:

**ВЕБ-ПРИЛОЖЕНИЕ ПЛАНИРОВАНИЯ ЗАДАЧ С УЧЁТОМ ИХ
МЕСТОПОЛОЖЕНИЯ И ВРЕМЕННЫХ ПАРАМЕТРОВ**

по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Веб-технологии»

Студент: _____ / Тютичкин Семен Владимирович, 221–329/
подпись *ФИО, группа*

Руководитель ВКР: _____ / Иванов Иван Иванович/
подпись *ФИО, уч. звание и степень*

Москва 2026

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Веб-технологии»

Тема ВКР	Веб-приложение планирования задач с учётом их местоположения и временных параметров
ПРАКТИЧЕСКИЙ РЕЗУЛЬТАТ	
Назначение	
Основные функции	1. Список функций
Используемые технологии и платформы	Перечисление технологий.

ВЫПОЛНЕНИЕ РАБОТЫ	
Решаемые задачи	1. Список задач
Состав технической документации	1. Список документации
Состав графической части	1. Список графической части

ПЛАН РАБОТЫ НАД ВКР

[illegible]

РУКОВОДИТЕЛЬ ОП:

«___»_____2026, _____ / Даньшина Марина Владимировна /
подпись *ФИО, уч. звание и степень*

РУКОВОДИТЕЛЬ ВКР:

«___»_____2026, _____ / Иванов Иван Иванович /
подпись *ФИО, уч. звание и степень*

СТУДЕНТ:

«___»_____2026, _____ / Тютичкин Семен Владимирович, 221–329/
подпись *ФИО, группа*

АННОТАЦИЯ

Наименование работы: Разработка веб-приложения для автоматизированного планирования маршрутов с учетом временных окон.

Цель работы: разработка веб-приложения для автоматизированного формирования оптимальной последовательности выполнения задач с учетом географического положения, временных окон и длительности выполнения.

Задачи для достижения цели: анализ предметной области и существующих программных решений; исследование алгоритмических подходов к маршрутизации с временными окнами; разработка архитектуры программной системы; реализация серверной части и алгоритма оптимизации маршрута; разработка пользовательского интерфейса с визуализацией маршрута на карте; интеграция с внешними картографическими сервисами; проведение тестирования; оценка производительности и масштабируемости.

Объект исследования: процесс планирования последовательности выполнения задач в городской среде с учетом пространственных и временных параметров.

Предмет исследования: алгоритмы маршрутизации и методы оптимизации порядка выполнения задач с временными окнами в условиях ограниченных вычислительных ресурсов веб-приложения.

Работа состоит из 3 глав, Введения, Заключения и Приложений. В работу включены рисунки и таблицы. Библиографический список включает источники, среди которых печатные и интернет-источники.

Во Введении описаны цель и задачи работы, объект и предмет исследования, актуальность и практическая значимость разработки.

В Первой главе проведен анализ предметной области, сформулирована постановка задачи, выполнен обзор и обоснование выбора технологий и платформ, проведен сравнительный анализ аналогичных программных решений.

Во Второй главе представлены модель данных разработанного приложения, проект пользовательского интерфейса и техническая реализация продукта, включая алгоритм оптимизации маршрута, серверную и клиентскую части, а также руководство по развертыванию и сопровождению.

В Третьей главе раскрыты технические аспекты внедрения и опытной эксплуатации: анализ соответствия реализованной системы требованиям, аспекты отказоустойчивости, надежности и производительности, процедуры развертывания и сопровождения.

В Заключении представлены выводы по поставленным задачам и оценка достигнутых результатов.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	9
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ	11
1.1 Исходное состояние объекта и характеристика предметной области	11
1.2 Постановка проблемы и формирование цели разработки	12
1.3 Обоснование выбора технологий и платформ	13
1.4 Анализ алгоритмических подходов к задаче маршрутизации с временными окнами	16
1.5 Обзор аналогов и сравнительный анализ	19
1.6 Выводы	20
2 РЕАЛИЗАЦИЯ	22
2.1 Модель данных	22
2.2 Архитектура системы	25
2.3 Алгоритм оптимизации маршрута	27
2.4 Тестирование	30
2.5 Интерфейс	34
2.6 Техническая реализация	38
2.7 Руководство пользователя и администратора	43
3 ВНЕДРЕНИЕ И ЭКСПЛУАТАЦИЯ	51
3.1 Анализ внедрения продукта	51
3.2 Аспекты отказоустойчивости, надежности и производительности	54
3.3 Внедрение и сопровождение программного продукта	58
ЗАКЛЮЧЕНИЕ	61
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	63
ПРИЛОЖЕНИЕ А	67
ПРИЛОЖЕНИЕ Б	73

ВВЕДЕНИЕ

В условиях цифровой трансформации городской среды и роста интенсивности деловой активности особую актуальность приобретает задача планирования времени и маршрутов перемещения. Увеличение количества задач, требующих личного присутствия, формируют потребность в инструментах автоматизированного планирования.

Существующие цифровые решения представлены либо менеджерами задач, ориентированными на учет перечня дел без учета пространственного фактора, либо навигационными сервисами, формирующими маршрут без учета длительности выполнения задач. В результате пользователи вынуждены комбинировать несколько приложений, вручную сопоставляя адреса, интервалы времени и порядок посещения точек. Подобный подход характеризуется высокой трудоемкостью и ростом вероятности ошибок при увеличении количества задач [1].

Объектом исследования является процесс планирования последовательности выполнения задач в городской среде с учетом пространственных и временных параметров.

Предметом исследования являются алгоритмы маршрутизации и методы оптимизации порядка выполнения задач с временными окнами в условиях ограниченных вычислительных ресурсов веб-приложения.

Целью выпускной квалификационной работы является разработка веб-приложения для автоматизированного формирования оптимальной последовательности выполнения задач с учетом географического положения, временных окон и длительности выполнения.

Для достижения поставленной цели необходимо решить следующие задачи:

— проанализировать предметную область и существующие программные решения;

- исследовать алгоритмические подходы к решению задач маршрутизации с временными окнами;
- разработать архитектуру программной системы;
- реализовать серверную часть с для вычисления оптимального маршрута;
- разработать пользовательский интерфейс для визуализации маршрута;
- реализовать интеграцию с внешними картографическими сервисами;
- провести тестирование разработанного решения;
- оценить производительность и масштабируемость системы.

Практическая значимость работы заключается в создании функционирующего программного продукта, позволяющего автоматизировать процесс планирования маршрутов для частных пользователей и субъектов малого бизнеса. Разработанное решение может быть использовано в деятельности выездных специалистов, курьеров и индивидуальных предпринимателей.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ

1.1 Исходное состояние объекта и характеристика предметной области

В настоящее время большинство пользователей решают задачу планирования маршрута вручную. Типичный сценарий включает формирование списка дел в менеджере задач, последующий поиск адресов в навигационном сервисе и самостоятельное определение порядка посещения точек [13]. При наличии временных окон (например, «с 9:00 до 12:00») пользователь вынужден дополнительно сопоставлять время перемещения и длительность выполнения задачи.

Существующие цифровые инструменты можно разделить на две группы:

- менеджеры задач (не учитывают пространственный фактор);
- навигационные сервисы (не работают с временными окнами и списками задач).

Таким образом, пользователь вынужден использовать несколько инструментов одновременно, что увеличивает трудозатраты и повышает вероятность ошибок [1].

Целевой аудиторией системы являются:

- частные пользователи, планирующие личные дела в городе;
- мастера и специалисты на выезде;
- курьеры малого бизнеса;
- индивидуальные предприниматели;
- диспетчеры, формирующие небольшие маршруты вручную.

Особенностью данной предметной области является необходимость одновременного учета:

- географического расположения объектов;

- временных ограничений;
- длительности выполнения задач;
- минимизации времени перемещения и ожидания.

Таким образом, предметная область представляет собой систему с признаками логистической оптимизации, однако в упрощённом масштабе, ориентированном на одного пользователя или малый бизнес.

1.2 Постановка проблемы и формирование цели разработки

Анализ исходного состояния показал, что основной проблемой является отсутствие доступного инструмента, объединяющего управление задачами и автоматическую маршрутизацию с учетом временных окон.

Ручной подход:

- требует значительных временных затрат;
- не гарантирует оптимальность маршрута;
- становится неэффективным при увеличении числа точек;
- не обеспечивает формализованного учета временных ограничений.

Задача рационального планирования маршрута с учетом временных ограничений относится к классу задач маршрутизации с временными окнами (Vehicle Routing Problem with Time Windows, VRPTW) — известной NP-трудной задаче комбинаторной оптимизации [11, 10, 12].

Целью разработки является создание веб-приложения, обеспечивающего автоматизированное формирование оптимального маршрута выполнения задач с учетом их географического положения, временных окон и длительности выполнения.

Назначение продукта заключается в цифровой трансформации процесса ручного планирования маршрута в автоматизированный процесс.

На основании цели сформирован следующий перечень функций:

- создание, редактирование и удаление задач;
- хранение географических координат;

- получение матрицы расстояний через внешние API;
- вычисление оптимальной последовательности выполнения задач;
- расчет времени прибытия, ожидания и завершения;
- визуализация маршрута на карте;
- экспорт маршрута в формате JSON.

Для обеспечения соответствия назначению определены следующие нефункциональные требования:

- время ответа сервера — не более 2 секунд при типовой нагрузке;
- поддержка современных браузеров;
- масштабируемость до 100 одновременных пользователей;
- устойчивость к повторным запросам внешних API за счет кэширования.

1.3 Обоснование выбора технологий и платформ

1.3.1 Выбор серверной платформы

В качестве языка для написания серверной части был выбран Golang[2].

Выбор языка Golang[2] обусловлен:

- статической типизацией, снижающей риск ошибок;
- встроенной моделью конкурентности на основе горутин [14];
- наличием практического опыта разработки.

1.3.2 Выбор СУБД и работы с геоданными

В качестве СУБД выбрана PostgreSQL[3].

Обоснование выбора:

- наличие геометрических индексов (GiST[4]);
- непрерывное промышленное применение с 1996 года; стабильный релизный цикл и соответствие стандарту SQL:2016 [3];

- наличием практического опыта разработки.

1.3.3 Выбор клиентских технологий

Для реализации пользовательского интерфейса был выбран React[5].

Причины выбора:

- компонентный подход к разработке;
- развитая экосистема;
- наличие поддержки картографических библиотек.

В проекте применяется TypeScript — статически типизированное надмножество JavaScript, что обеспечивает раннее обнаружение ошибок на этапе компиляции. Эмпирическое исследование 604 проектов на GitHub подтверждает, что TypeScript-проекты демонстрируют значительно более высокое качество и понятность кода по сравнению с аналогами на JavaScript [25].

1.3.4 Выбор картографического API

Для геокодирования адресов и визуализации маршрута выбран Яндекс Карты JavaScript API 2.1. В таблице 1 представлено сравнение доступных картографических решений.

Таблица 1 – Сравнение картографических API

Критерий	Яндекс Карты JS API	Google Maps JS API	OpenStreetMap / Leaflet
Геокодер на русском языке	Да, включая топонимику РФ	Да	Ограниченно (Nominatim)
Бесплатный лимит (геокодер)	1 000 за пр./сутки	200 \$ кредит/мес.	Без ограничений
MultiRoute для матрицы расстояний	Да (встроено)	Нет (отдельный API)	Нет (требуется OSRM [20])
Работа в России	Без ограничений	Может быть недоступен	Без ограничений

По совокупности критериев таблицы 1 для целевой аудитории (российские пользователи) выбраны Яндекс Карты: единственный из рассмотренных вариантов, обеспечивающий поддержку российской топонимики, встроенный мультимаршрутизатор для построения матрицы расстояний и гарантированную доступность в российском сетевом сегменте.

1.3.5 Итоговый технологический стек

В таблице 2 приведен обоснованный итоговый технологический стек.

Таблица 2 – Итоговый технологический стек разрабатываемой системы

Уровень	Технология	Обоснование
Серверная часть	Go 1.23 + Gin [22]	Нативная компиляция, модель конкурентности goroutine, минимальные накладные расходы
База данных	PostgreSQL 16 + PostGIS [21]	Реляционность, геоиндексы GiST, зрелость платформы
Клиентская часть	React 18 + TypeScript [23]	Компонентный подход, статическая типизация, развитая экосистема
Сборка фрон-тенда	Vite 6 [24]	HMR, оптимизированная продакшен-сборка
Картографический API	Яндекс Карты JS API 2.1	Русскоязычный геокодер, встроенный мультимаршрутизатор
Контейнеризация	Docker Compose [6]	Воспроизводимость среды, автоматическое применение миграций

1.4 Анализ алгоритмических подходов к задаче маршрутизации с временными окнами

Задача, решаемая в данной работе, относится к классу VRPTW, рассмотренному в предыдущем разделе, — обобщению классической задачи коммивояжера (TSP) [18] с дополнительными временными ограничениями.

1.4.1 Формальная постановка задачи

Пусть задан граф $G = (V, E)$, где $V = \{v_0, v_1, \dots, v_n\}$ — множество узлов (задач пользователя), E — множество дуг с матрицей расстояний d_{ij} и матрицей времен перемещения t_{ij} .

Каждый узел v_i ($i \geq 1$) характеризуется:

- временным окном $[a_i, b_i]$ — допустимым интервалом начала обслуживания;
- длительностью обслуживания s_i (минуты).

Если маршрутный агент прибывает в узел v_i в момент $\tau_i < a_i$, он ожидает до момента a_i ; если $\tau_i > b_i - s_i$ — обслуживание начать невозможно (нарушение ограничения).

Цель: найти порядок посещения всех узлов, минимизирующий суммарное время маршрута $T = \sum_i t_{ij} + \sum_i w_i$, где $w_i = \max(0, a_i - \tau_i)$ — время ожидания в узле v_i , при соблюдении условий $\tau_i \leq b_i - s_i$ для всех i .

1.4.2 Сравнение алгоритмических подходов

В таблице 3 приведен сравнительный анализ основных подходов к решению VRPTW применительно к задачам данной работы.

Таблица 3 – Сравнение алгоритмических подходов к задаче VRPTW

Подход	Сложность	Оптимальность	Применимость
Полный перебор (brute force)	$O(n!)$	Да	Только при $n \leq 10$; неприемлем для веб-приложения
Динамическое программирование (алг. Хелда–Карпа [18])	$O(2^n \cdot n^2)$	Да	До $n \approx 20$; требует значительного объема памяти
Жадная эвристика NNH-TW (ближайший сосед с временными окнами)	$O(n^2)$	Нет	Веб-приложение; $n < 50$; предсказуемое время ответа
Муравьиный алгоритм (ACO) [10, 27]	$O(iter \cdot n^2 \cdot m)$	Нет	Средний масштаб; сложная настройка параметров
Генетические алгоритмы (GA) [19]	$O(gen \cdot pop \cdot n)$	Нет	Большой масштаб; длительное время вычислений

Здесь n — количество узлов маршрута, m — количество муравьев (агентов), $iter$ — число итераций, gen — число поколений, pop — размер популяции.

1.4.3 Обоснование выбора эвристики NNH-TW

Для разрабатываемого веб-приложения, ориентированного на частных пользователей и малый бизнес, был выбран алгоритм «ближайшего соседа с временными окнами» (NNH-TW, Nearest Neighbour Heuristic with Time Windows) по следующим причинам:

— **Квадратичная сложность** $O(n^2)$: при $n = 30$ задачах алгоритм выполняет не более $30^2 = 900$ элементарных сравнений, что на несколько порядков меньше задержки, вносимой получением матрицы расстояний от внешнего API (экспериментально подтверждено в разделе «Производительность алгоритма оптимизации»);

— **Предсказуемость**: время работы линейно зависит от числа задач, что гарантирует выполнение нефункционального требования по времени ответа (не более 2 секунд);

— **Качество решений**: жадные конструктивные эвристики обеспечивают приемлемое качество решений при низких вычислительных затратах [26], что достаточно для целей планирования личных задач и курьерских маршрутов малого масштаба;

— **Расширяемость**: реализованный интерфейс `Optimizer` позволяет заменить алгоритм на более мощный (например, АСО или GA) без изменения остального кода системы.

1.5 Обзор аналогов и сравнительный анализ

В качестве аналогов были рассмотрены Relog [7], Яндекс Маршрутизация [8] и Spoke [9].

Для оценки выбраны критерии:

- целевой сегмент;
- модель оплаты;
- поддержка пешеходных маршрутов;
- наличие русского языка;

— ориентация на малый бизнес и частных пользователей.

Из таблицы 4 следует, что все рассмотренные аналоги ориентированы на средний и крупный бизнес и работают по платной модели (подписка или оплата за запрос). Ни один из них не предусматривает бесплатного тарифа, ориентированного на частного пользователя, что исключает их практическое применение для целевой аудитории разрабатываемой системы.

Таблица 4 – Сравнительный анализ аналогов

Критерии сравнения	Relog	Яндекс Маршрути- зация	Spoke	Проектируемое решение
Целевой сегмент	Средний и крупный бизнес	Средний и крупный бизнес	Индивидуальные курьеры	Частный пользователь и малый бизнес
Модель оплаты	Подписка	За каждый запрос	Условно-бесплатный	Бесплатный
Работает с пешеходными маршрутами	Нет	Да	Нет	Да
Наличие русского языка	Да	Да	Нет	Да

1.6 Выводы

Проведен анализ предметной области, выявлены ограничения существующих инструментов и обоснована актуальность разработки системы планирования маршрутов с учетом временных окон. Сформулирована цель

и назначение продукта, определен перечень функций и нефункциональных требований.

Проведен сравнительный анализ алгоритмических подходов к задаче VRPTW: полный перебор, динамическое программирование, жадные эвристики и метаэвристики. Обоснован выбор алгоритма NNH-TW с квадратичной сложностью $O(n^2)$ как наиболее подходящего для целевого масштаба системы ($n < 50$) и требований по времени ответа.

На основании сравнительного анализа технологий выбран стек разработки, обеспечивающий требуемую производительность и масштабируемость системы. Проведенный анализ аналогов подтвердил целесообразность создания специализированного решения, ориентированного на частных пользователей и малый бизнес.

2 РЕАЛИЗАЦИЯ

2.1 Модель данных

2.1.1 Общая характеристика структуры хранения данных

На рисунке 1 представлена логическая модель данных разрабатываемого веб-приложения.

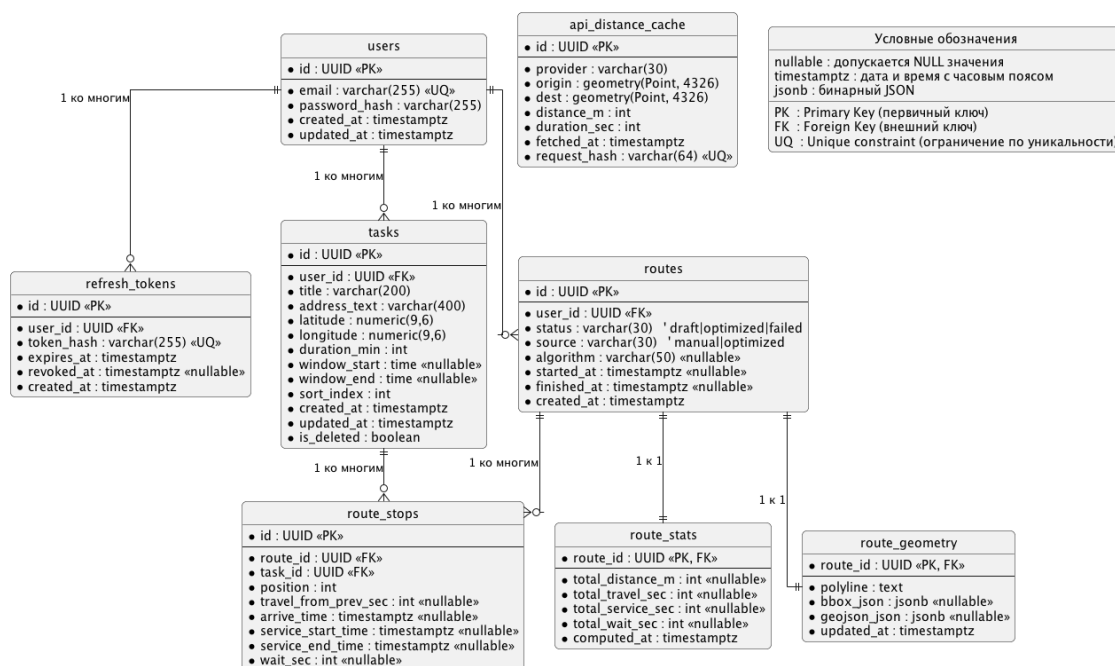


Рисунок 1 – Модель данных

Модель данных реализована на основе реляционного подхода и ориентирована на использование системы управления базами данных PostgreSQL.

Структура базы данных отражает предметную область приложения и включает следующие функциональные группы сущностей:

- хранение информации о пользователях;
- хранение задач с географической привязкой;
- хранение маршрутов;
- хранение результатов вычислений ;
- вспомогательные данные для интеграции с внешними API.

Такое разделение позволяет логически отделить пользовательские данные от результатов алгоритмической обработки и служебной информации.

2.1.2 Сущности, связанные с пользователями

Сущность `users` предназначена для хранения учетных записей пользователей. Первичный ключ `id` используется для установления связей со всеми зависимыми таблицами. Поле `email` выполняет роль логина и имеет ограничение уникальности, что исключает дублирование учетных записей. Пароль хранится в виде хэшированного значения, что обеспечивает безопасность аутентификации.

Сущность `refresh_tokens` используется для поддержки механизма продления авторизации. Каждый токен связан с конкретным пользователем через внешний ключ `user_id`. Хранение токена в виде хэша предотвращает его компрометацию в случае несанкционированного доступа к базе данных.

2.1.3 Сущность задач

Сущность `tasks` отражает основную предметную сущность системы — задачу, подлежащую выполнению.

Каждая запись содержит:

- связь с владельцем задачи (`user_id`);
- текстовое описание адреса (`address_text`);
- географические координаты (`latitude`, `longitude`) в которых хранится широта и долгота соответственно;
- длительность выполнения (`duration_min`);
- границы временного окна (`window_start`, `window_end`);
- индекс сортировки (`sort_index`) для отображения порядка в пользовательском интерфейсе.

2.1.4 Сущности маршрутов

Сущность `routes` описывает факт построения маршрута для конкретного пользователя. Она содержит сведения о статусе вычисления, источнике

формирования последовательности (ручной или оптимизированный) и времени создания.

Структура маршрута детализируется в сущности `route_stops`, где каждая запись соответствует отдельной точке маршрута. Поле `position` определяет порядок следования, а временные поля (`arrive_time`, `service_start_time`, `service_end_time`) позволяют зафиксировать результат работы алгоритма с учетом временных окон.

Таким образом, разделение сущностей `routes` и `route_stops` обеспечивает нормализацию данных и упрощает обработку маршрутов переменной длины.

2.1.5 Хранение агрегированных показателей

Агрегированные характеристики маршрута вынесены в отдельную сущность `route_stats`. К таким показателям относятся суммарная длина маршрута, время перемещения, время выполнения задач и суммарное ожидание.

Выделение агрегированных данных в отдельную таблицу позволяет:

- уменьшить нагрузку на основную таблицу маршрутов;
- обновлять статистику независимо от структуры маршрута;
- отделить расчетные показатели от исходных данных.

2.1.6 Геометрия маршрута

Сущность `route_geometry` предназначена для хранения пространственного представления маршрута, используемого при визуализации на карте. Поле `polyline` либо `geojson_json` содержит геометрическое описание линии маршрута, полученное из внешнего API.

Разделение логической структуры маршрута и его геометрического представления позволяет обновлять визуальные данные без изменения структуры остановок.

2.1.7 Кэширование данных внешних API

Сущность `api_distance_cache` используется для хранения ранее полученных значений расстояния и времени перемещения между парами точек.

Наличие кэша снижает количество обращений к внешним сервисам, уменьшает задержки при повторной оптимизации и повышает устойчивость системы при ограничениях со стороны API-провайдера.

2.1.8 Вывод

Представленная модель данных обеспечивает логическое разделение сущностей и позволяет хранить как исходные данные пользователя, так и результаты алгоритмической обработки.

Структура базы данных соответствует функциональным требованиям системы и обеспечивает возможность дальнейшего расширения без существенной переработки архитектуры.

2.2 Архитектура системы

2.2.1 Общие принципы

Серверная часть приложения построена по принципам чистой архитектуры (Clean Architecture), предложенной Р. Мартином [17]. Ключевой принцип: зависимости направлены исключительно от внешних слоев (HTTP-обработчики, репозитории) к внутренним (доменные сущности и варианты использования), но не наоборот. Это обеспечивает независимость бизнес-логики от конкретных фреймворков, СУБД и транспортных протоколов.

2.2.2 Слои архитектуры

Приложение организовано в четыре слоя, представленных в таблице 5.

Таблица 5 – Слои чистой архитектуры серверной части

Слой	Пакет	Назначение
Доменные сущности	<code>internal/domain</code>	Структуры данных (Task, Route, User); без внешних зависимостей
Интерфейсы (порты)	<code>internal/domain</code>	Интерфейсы репозитория и оптимизатора; определяют контракт без реализации
Варианты использования	<code>internal/domain</code>	Бизнес-логика: создание задач, запуск оптимизации, управление сессией
Адаптеры	<code>internal/api/http</code> и <code>internal/platform</code>	HTTP-обработчики и SQL-репозитории; зависят от внешних библиотек

2.2.3 Паттерн замены алгоритма

Ключевым архитектурным решением является интерфейс `Optimizer`, определенный в пакете `internal/domain/route/gate/`:

— метод `Optimize(ctx, graph, startTimeMins)` принимает граф задач и начальное время, возвращает порядок обхода и агрегированную статистику;

— конкретная реализация `NearestNeighborTW` регистрируется в модуле сборки зависимостей (`internal/app/app.go`);

— для замены алгоритма (например, на муравьиный или генетический) достаточно зарегистрировать новую реализацию интерфейса, не изменяя HTTP-обработчики, репозитории или доменные сущности.

2.2.4 Вывод

Применение принципов чистой архитектуры позволило разделить бизнес-логику и инфраструктурный код, упростить тестирование (слой `story` тестируется через `mock-репозитории`) и обеспечить возможность замены алгоритма оптимизации без рефакторинга смежных компонентов.

2.3 Алгоритм оптимизации маршрута

2.3.1 Описание алгоритма

Для вычисления оптимального порядка посещения задач реализован алгоритм «ближайшего соседа с временными окнами» (Nearest Neighbour Heuristic with Time Windows, NNH-TW).

Алгоритм является жадной конструктивной эвристикой: на каждом шаге выбирается ближайшая еще не посещенная точка среди тех, временные ограничения которых позволяют успеть выполнить задачу.

2.3.2 Псевдокод алгоритма

Листинг 2.1 – Псевдокод алгоритма NNH-TW

```
1 Вход: граф  $G(V, E)$ , startTimeMins  
2 Выход: порядок обхода order[], агрегированная статистика  
  
3 1.  $cur \leftarrow \text{startNode}(G)$  // узел с наиболее  
    $\rightarrow$  ранним временным окном  
4 2. visited[cur]  $\leftarrow$  true  
5 3. order  $\leftarrow$  [cur]  
6 4. currentTime  $\leftarrow$  startTimeMins + duration[cur]  
  
7 5. Пока |order| < |V|:
```

```

8 // Проход 1: ближайший достижимый в пределах окна
9 a. next ← -1; best ← ∞
10 b. Для каждого i ∉ visited:
11     arrivalMins ← currentTime + travelSec[cur][i] / 60
12     Если feasible(i, arrivalMins) И travelSec[cur][i] <
        ↪ best:
13         best ← travelSec[cur][i]; next ← i
14 // Проход 2: резервный — ближайший без учета окна
15 c. Если next = -1:
16     best ← ∞
17     Для каждого i ∉ visited:
18         Если travelSec[cur][i] < best:
19             best ← travelSec[cur][i]; next ← i
20 d. arrivalMins ← currentTime + travelSec[cur][next] / 60
21 e. wait ← max(0, windowStart[next] - arrivalMins)
22 f. currentTime ← arrivalMins + wait + duration[next]
23 g. visited[next] ← true; order.append(next); cur ← next
24 6. Вернуть order и статистику

25 feasible(node i, arrivalMins):
26     Если windowEnd[i] = -1: вернуть true // ограничений
        ↪ нет
27     Вернуть arrivalMins ≤ windowEnd[i] - duration[i]

```

2.3.3 Трассировка на числовом примере

Рассмотрим пример с тремя задачами для иллюстрации работы алгоритма. Исходные данные приведены в таблице 6, матрица времен перемещения — в таблице 7.

Таблица 6 – Исходные данные задач для трассировки

Узел	Адрес	Временное окно	Длительность
v_0	Офис (начало)	08:00–09:00	30 мин
v_1	Точка А	10:00–12:00	20 мин
v_2	Точка Б	09:00–10:00	15 мин

Таблица 7 – Матрица времен перемещения (минуты)

Из / В	v_0	v_1	v_2
v_0	—	40	15
v_1	40	—	20
v_2	15	20	—

Трассировка алгоритма при начале работы в 08:00 ($\text{startTimeMins} = 480$):

а) **Выбор стартового узла.** Узел v_0 имеет наиболее раннее окно (08:00)
 $\rightarrow \text{cur} = 0, \text{currentTime} = 480 + 30 = 510$ (08:30 + 30 мин выполнения = 09:30
 $\rightarrow$ мин 510).

б) **Шаг 1. Проход 1.**

— v_1 : прибытие = $510 + 40 = 550$ (09:10). Окно v_1 : 600–720 (10:00–12:00). Условие: $550 \leq 720 - 20 = 700 \rightarrow \text{feasible}$. $t = 40$ мин.

— v_2 : прибытие = $510 + 15 = 525$ (08:45). Окно v_2 : 540–600 (09:00–10:00). Условие: $525 \leq 600 - 15 = 585 \rightarrow \text{feasible}$. $t = 15$ мин.

— v_2 ближе ($15 < 40$) $\rightarrow \text{next} = 2$.

Ожидание: $\max(0, 540 - 525) = 15$ мин. $\text{currentTime} = 525 + 15 + 15 = 555$ (09:15).

в) **Шаг 2. Проход 1.**

— v_1 : прибытие = $555 + 20 = 575$ (09:35). Условие: $575 \leq 700 \rightarrow \text{feasible}$. $\text{next} = 1$.

Ожидание: $\max(0, 600 - 575) = 25$ мин. $\text{currentTime} = 575 + 25 + 20 = 620$ (10:20).

Итоговый порядок: $v_0 \rightarrow v_2 \rightarrow v_1$. Суммарное ожидание: $15 + 25 = 40$ мин.

2.3.4 Оценка вычислительной сложности

Алгоритм выполняет n итераций основного цикла, на каждой из которых просматривает до n узлов. Таким образом, сложность составляет $O(n^2)$.

При $n = 30$ задачах алгоритм выполняет не более $30 \times 30 = 900$ операций сравнения. Результаты бенчмаркинга, приведённые в таблице 12, показывают, что при $n \leq 30$ время работы алгоритма составляет менее 1 мс, что на несколько порядков меньше задержки, вносимой получением матрицы расстояний от внешнего API.

2.3.5 Вывод

Алгоритм NNH-TW обеспечивает приемлемое для практических задач качество решений [26] при гарантированном $O(n^2)$ времени работы, что полностью соответствует нефункциональным требованиям системы по времени ответа. Двухпроходная схема (Pass 1 / Pass 2) гарантирует посещение всех задач даже при нарушении временных ограничений.

2.4 Тестирование

2.4.1 Стратегия тестирования

В проекте реализована многоуровневая стратегия тестирования, охватывающая как серверную, так и клиентскую части приложения.

Применяемые виды тестирования:

— **Модульные тесты** (unit tests) — проверка отдельных функций и методов в изоляции от внешних зависимостей;

— **Тесты с заглушками** (mock-тесты) — тестирование вариантов использования (story) через интерфейсы с подмененными реализациями репозитория;

— **Тесты компонентов** (component tests) — проверка React-компонентов через `@testing-library/react` с симуляцией пользовательских действий;

— **Тесты API-клиента** — перехват HTTP-запросов через Mock Service Worker (MSW) и проверка логики обновления токена.

Для написания тестов серверной части используется библиотека `github.com/stretchr/testify`: пакет `assert` для нестрогих проверок и пакет `require` для немедленной остановки теста при критическом сбое.

2.4.2 Тестирование алгоритма оптимизации

Наиболее критичным компонентом с точки зрения корректности является алгоритм NNH-TW. Для его проверки разработано 12 тест-кейсов, охватывающих все ветви алгоритма. Перечень тест-кейсов приведен в таблице 8.

Таблица 8 – Тест-кейсы алгоритма NNH-TW

Название теста	Проверяемое условие
TestOptimize_TwoNodes	Граф из 2 узлов без временных ограничений; оба посещены ровно один раз
TestOptimize_ThreeNodes	3 узла, ближайший сосед выбирается корректно (порядок 0→1→2)
TestOptimize_TimeWindow	Стартовый и финиш выбирается по наиболее раннему временному окну
TestOptimize_WaitBefore	Ожидание перед открытием окна (TotalWaitSec > 0)
TestOptimize_Fallback	Pass 2 узла выбирается без учета окна при отсутствии feasible-узлов
TestOptimize_Aggregate	Корректность агрегатных показателей (расстояние, время, сервис)
TestOptimize_OneNode	Граф из одного узла; order = [0]
TestOptimize_StartTime	Начало работы в 08:00; оба узла посещены
TestOptimize_Symmetric	Симметричная матрица из 4 узлов; каждый посещен ровно один раз
TestOptimize_Unreachable	«Недосяжимые» пары (99 999 сек); все узлы посещены через Pass 2
TestOptimize_EmptyGraph	Пустой граф; order = [] без ошибок
TestFeasible_* (5 случаев)	Граничные значения функции feasible(): нет ограничений, в пределах окна, до открытия, после закрытия, на границе

2.4.3 Тестирование бизнес-логики серверной части

Варианты использования (use cases) тестируются с использованием mock-репозитория, генерируемого пакетом `testify/mock`. Такой подход позволяет тестировать бизнес-логику в изоляции от базы данных.

Покрываются следующие компоненты:

- `auth/story` — регистрация, вход, выход, обновление токена;
- `task/story` — создание, редактирование, удаление задачи, проверка принадлежности задачи пользователю;
- `route/story` — сохранение оптимизированного маршрута, получение маршрута по идентификатору.

2.4.4 Тестирование клиентской части

Клиентская часть тестируется с использованием стека: `vitest` — тестовый раннер, `@testing-library/react` — тестирование компонентов, `msw` (Mock Service Worker) — перехват и подмена HTTP-запросов в тестах.

Ключевые тест-кейсы клиентской части:

- Логика обновления токена: при получении ответа 401 запускается ровно один запрос обновления (дедупликация через `shared Promise`); после успешного обновления исходный запрос повторяется;
- Утилита построения матрицы расстояний: корректное формирование $n \times (n - 1)$ запросов к мультимаршрутизатору Яндекс Карт.

2.4.5 Вывод

Реализованная стратегия тестирования охватывает все критические компоненты системы: алгоритм оптимизации (12 тест-кейсов), бизнес-логику серверной части (mock-тесты use cases) и клиентский API-клиент (MSW-тесты). Использование интерфейсов и `dependency injection` обеспечи-

вает возможность полноценного тестирования бизнес-логики без обращения к реальной базе данных.

2.5 Интерфейс

На рисунках 2, 3, 4, 5, 6 представлены примерные макеты разрабатываемого приложения:

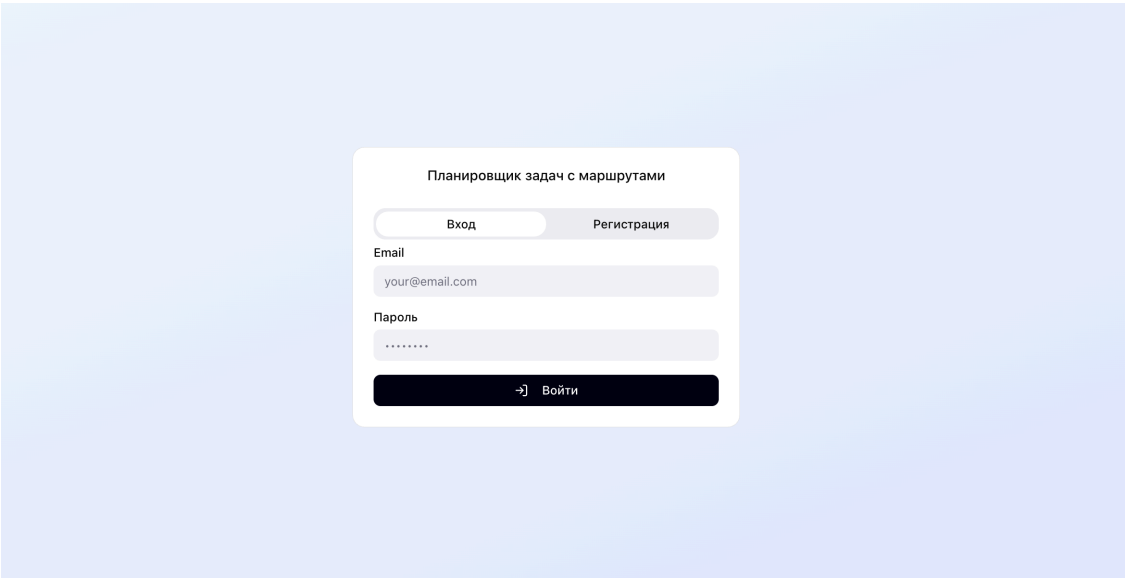


Рисунок 2 – Макет страницы с авторизацией

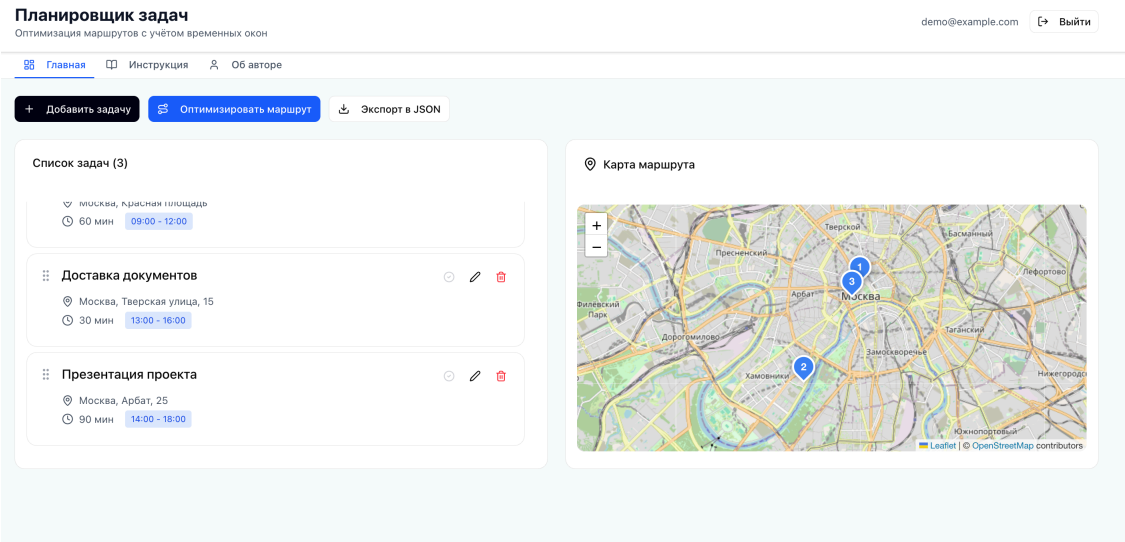


Рисунок 3 – Макет главной страницы приложения

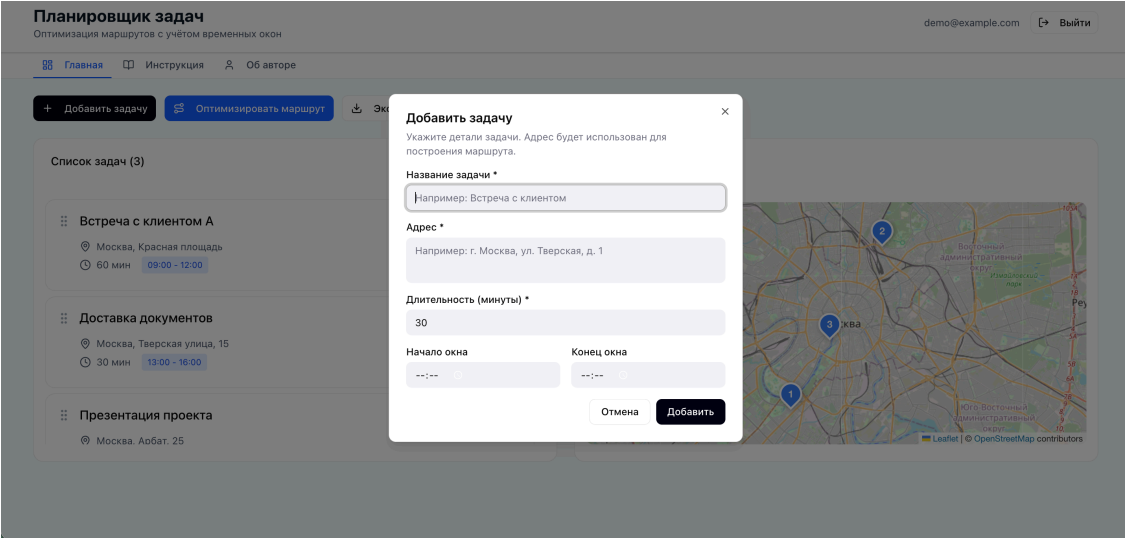


Рисунок 4 – Макет попапа для создания новой задачи

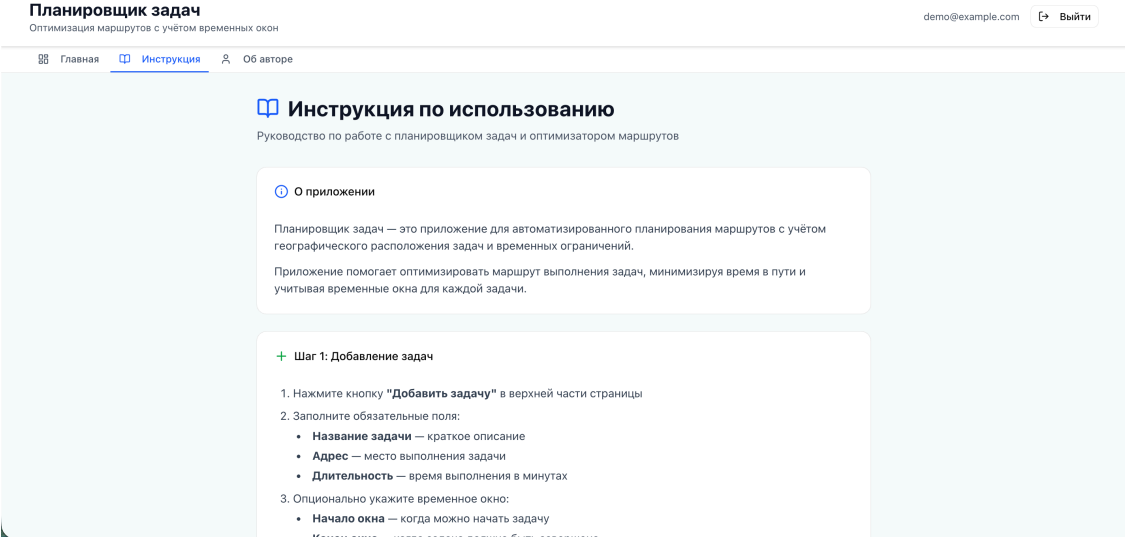


Рисунок 5 – Макет страницы с инструкций к приложению

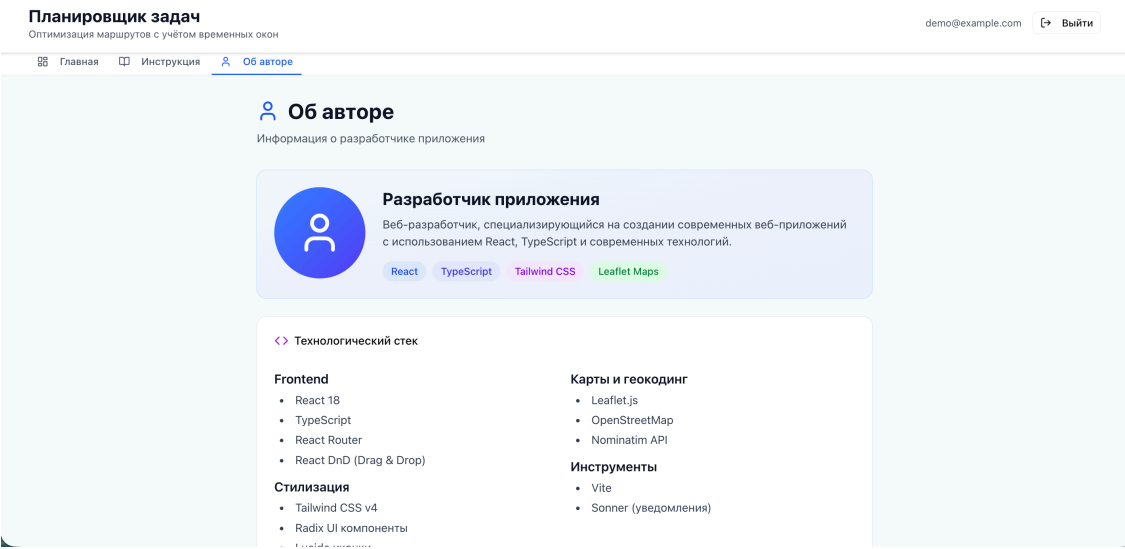


Рисунок 6 – Макет страницы «Об авторе»

2.5.1 Общая логика построения интерфейса

Интерфейс разрабатывался исходя из основной задачи системы — планирования последовательности выполнения задач с учетом их географического положения и временных ограничений.

При проектировании было важно, чтобы пользователь одновременно видел исходные данные и результат их обработки. Поэтому на главной странице (рисунок 3) список задач и карта маршрута размещены в пределах одного экрана. Такое решение позволяет сразу оценивать влияние изменений в списке задач на итоговый маршрут.

2.5.2 Организация рабочей области

Рабочая область разделена на две функциональные зоны:

- список задач;
- карта с визуализацией маршрута.

Список задач представлен в виде отдельных карточек. Карточный формат обеспечивает визуальное разграничение элементов списка: каждая задача представлена отдельным блоком с фиксированным набором полей, что снижает когнитивную нагрузку при одновременном отображении нескольких задач.

Карта размещена справа, что обеспечивает постоянную визуальную связь между порядком задач и их пространственным расположением. Пользователь может сопоставить маршрут с реальным расположением точек без дополнительных переходов между страницами.

2.5.3 Организация ввода данных

Добавление новой задачи реализовано через модальное окно (рисунок 4).

Модальный формат выбран для изоляции процесса ввода данных. При открытии формы внимание пользователя концентрируется исключительно на заполнении полей. После закрытия окна пользователь возвращается в основную рабочую область без перезагрузки страницы.

Поля формы сгруппированы логически: название, адрес, длительность, временное окно. Такая последовательность соответствует естественному порядку формулирования задачи.

2.5.4 Навигационная структура

Навигация организована через верхнюю панель (рисунки 5, 6).

В интерфейсе предусмотрены следующие разделы:

- главная страница — работа с задачами и маршрутом;
- инструкция — описание принципа работы системы;
- информация об авторе.

Количество разделов минимально, поскольку приложение ориентировано на решение одной основной задачи. Это позволяет избежать перегруженности навигации.

2.5.5 Поддержка пользовательского контроля

В интерфейсе реализована возможность изменения порядка задач методом перетаскивания.

Даже при наличии автоматической оптимизации пользователь сохраняет возможность ручного управления последовательностью выполнения. Это обеспечивает соответствие принципу пользовательского контроля: оператор может скорректировать автоматически сформированный порядок без повторного запуска алгоритма.

2.5.6 Вывод

Принятые решения по организации интерфейса обеспечивают логичную структуру взаимодействия, наглядность отображения пространственных данных и соответствие функциональным требованиям системы.

2.6 Техническая реализация

2.6.1 Состав и объем программного кода

Разработанное приложение состоит из двух независимых программных компонентов: серверной части на языке Go и клиентской части на языке TypeScript с использованием фреймворка React.

Серверная часть насчитывает 40 исходных файлов на языке Go, организованных в соответствии с принципами чистой архитектуры. Код распределен по следующим функциональным пакетам:

- `cmd/api` — точка входа в приложение (1 файл);
- `internal/config` — загрузка конфигурации из переменных окружения (1 файл);
- `internal/app` — сборка зависимостей (1 файл);
- `internal/domain` — бизнес-логика предметной области: сущности, интерфейсы репозитория и варианты использования для модулей аутентификации, задач, маршрутов и пользователей (18 файлов, включая тесты);
- `internal/api/http` — HTTP-обработчики, промежуточное программное обеспечение, маршрутизатор и объекты передачи данных (8 файлов, включая тесты);
- `internal/platform/postgres` — реализации репозитория и подключение к базе данных (5 файлов);
- `internal/platform/distance` — провайдер матрицы расстояний с реализацией на основе OSRM (2 файла);
- `migrations` — SQL-миграции схемы базы данных (2 файла).

Клиентская часть содержит свыше 90 исходных файлов TypeScript и TSX. Из них около 30 файлов реализуют прикладную логику и компоненты страниц, остальные относятся к библиотеке пользовательского интерфейса на основе `shadcn/ui` и `Radix UI`. Структура клиентской части включает:

- корневые компоненты приложения и конфигурация маршрутизации (3 файла);
- API-клиент с логикой обновления токена (2 файла);
- контекст аутентификации и хук `useAuth()` (1 файл);
- типы TypeScript для доменных сущностей (3 файла);
- утилиты загрузки Яндекс Карт и построения матрицы расстояний (2 файла);
- компоненты страниц и функциональные компоненты (11 файлов);
- компоненты UI-библиотеки (более 60 файлов).

Наиболее значимые фрагменты кода приведены в приложениях: реализация алгоритма оптимизации маршрута — в приложении 3.1, обработчики HTTP-запросов для маршрутов — в приложении 3.2, клиент API с логикой обновления токена — в приложении 3.3, утилита построения матрицы расстояний — в приложении 3.4.

2.6.2 Используемые библиотеки серверной части

В серверной части используются следующие сторонние библиотеки Go:

- `github.com/gin-gonic/gin` (v1.10.0) — HTTP-фреймворк для построения REST API; обеспечивает маршрутизацию запросов, группировку эндпоинтов, обработку ошибок и привязку JSON-тела запроса к структурам данных;
- `github.com/gin-contrib/cors` (v1.7.2) — промежуточное программное обеспечение для настройки политики CORS (Cross-Origin Resource Sharing);

— `github.com/golang-jwt/jwt/v5` (v5.2.1) — генерация и верификация JSON Web Token для реализации механизма аутентификации без сохранения состояния на сервере;

— `github.com/jackc/pgx/v5` (v5.6.0) — низкоуровневый драйвер PostgreSQL для Go с поддержкой пула соединений; используется вместо стандартного `database/sql` ввиду более высокой производительности и поддержки специфичных типов PostgreSQL;

— `github.com/google/uuid` (v1.6.0) — генерация уникальных идентификаторов UUID версии 4 для первичных ключей всех таблиц;

— `golang.org/x/crypto` (v0.27.0) — функции криптографии, в частности `bcrypt` для хэширования паролей пользователей и токенов обновления;

— `github.com/stretchr/testify` (v1.9.0) — набор вспомогательных функций для написания тестов: утверждения `assert/require` и создание заглушек через пакет `mock`.

Для управления миграциями схемы базы данных используется инструмент `migrate/migrate` (v4.17.1), запускаемый автоматически при старте Docker-контейнера.

2.6.3 Используемые библиотеки клиентской части

В клиентской части задействованы следующие основные библиотеки:

— `react` и `react-dom` (v18.3.1) — базовый фреймворк для построения компонентного пользовательского интерфейса;

— `react-router` (v7.13.0) — декларативная клиентская маршрутизация; используется для разделения приложения на страницы без перезагрузки браузера;

— `vite` (v6.3.5) с плагином `@vitejs/plugin-react` — инструмент сборки; обеспечивает быстрое обновление модулей в режиме разработки (HMR) и оптимизированную сборку для продакшена;

- `tailwindcss` (v4) — утилитарный CSS-фреймворк; используется для стилизации компонентов без написания отдельных файлов стилей;
- пакеты `@radix-ui/*` (более 20 пакетов) — безголовые примитивы доступного пользовательского интерфейса (диалоги, всплывающие подсказки, вкладки, аккордеон и др.); служат основой для компонентов `shadcn/ui`;
- `react-hook-form` (v7.55.0) — управление состоянием форм с минимальным количеством перерисовок; применяется в форме создания задачи;
- `zod` — библиотека декларативной валидации схем данных; используется совместно с `react-hook-form`;
- `react-dnd` и `react-dnd-html5-backend` — реализация перетаскивания элементов (`drag-and-drop`) для ручного упорядочивания задач;
- `sonner` (v2.0.3) — компонент всплывающих уведомлений (`toast`);
- `lucide-react` — библиотека SVG-иконок;
- `date-fns` — утилиты для форматирования дат и работы со временем.

Для тестирования клиентской части используется стек: `vitest` (v3.2.4) как тестовый раннер, `@testing-library/react` для тестирования компонентов, `@testing-library/user-event` для симуляции пользовательских действий, а также `msw` (Mock Service Worker) для перехвата и подмены HTTP-запросов в тестах.

Картографический функционал реализован на основе Яндекс Карт JavaScript API 2.1, загружаемого динамически через `<script>`-тег при первом обращении к карте. Данный API предоставляет геокодер, механизм мультимаршрутизации для построения матрицы расстояний и инструменты визуализации (маркеры, полилинии, управление видимой областью карты).

2.6.4 Требования к техническим средствам

Для развертывания и эксплуатации приложения предъявляются следующие требования к техническим средствам.

Для серверной части:

- процессор: одноядерный с тактовой частотой не ниже 1 ГГц (рекомендуется двухъядерный и более);
- оперативная память: не менее 512 МБ (рекомендуется 1 ГБ и более);
- дисковое пространство: не менее 1 ГБ для хранения образов Docker и данных PostgreSQL;
- сетевой интерфейс: соединение с интернетом для обращения к внешним сервисам (Яндекс Карты, OSRM).

Для клиентской части (рабочее место пользователя):

- устройство с современным веб-браузером и поддержкой JavaScript;
- соединение с интернетом для загрузки Яндекс Карт и обращения к API.

2.6.5 Требования к программному обеспечению

Для развертывания серверной части с использованием Docker необходимо наличие:

- операционной системы Linux, macOS или Windows с поддержкой контейнеризации;
- Docker Engine версии 24.0 и выше;
- Docker Compose версии 2.0 и выше.

При запуске серверной части без Docker дополнительно требуются:

- Go версии 1.23 и выше;
- PostgreSQL версии 16 с расширением PostGIS версии 3.4;
- утилита `migrate` для применения миграций.

Для сборки и запуска клиентской части в режиме разработки необходимы:

- Node.js версии 18 и выше;
- менеджер пакетов `pnpm` версии 8 и выше (либо `npm` версии 9 и выше).

На стороне пользователя поддерживаются следующие браузеры: Google Chrome версии 90 и выше, Mozilla Firefox версии 88 и выше, Apple Safari версии 14 и выше, Microsoft Edge версии 90 и выше.

2.6.6 Вывод

Программный код приложения структурирован в соответствии с принципами чистой архитектуры: серверная часть содержит 40 файлов на языке Go, клиентская — свыше 90 файлов на языке TypeScript.

Выбор библиотек обусловлен функциональными требованиями и широкой распространенностью соответствующих решений в профессиональном сообществе. Инфраструктура на основе Docker обеспечивает воспроизводимость развертывания и независимость от конфигурации целевой системы.

2.7 Руководство пользователя и администратора

2.7.1 Общие сведения о развертывании

Приложение состоит из двух независимо запускаемых компонентов: серверной части (Go API + PostgreSQL) и клиентской части (React SPA). Серверная часть распространяется в виде Docker-образов и предназначена для запуска на сервере или локальной машине. Клиентская часть запускается в веб-браузере и не требует установки дополнительного программного обеспечения на стороне пользователя.

Поддерживаются следующие платформы для развертывания серверной части: Linux (рекомендуется), macOS и Windows с установленным Docker Engine.

2.7.2 Установка и настройка серверной части

2.7.2.1 Развертывание с использованием Docker Compose

Рекомендуемым способом развертывания является использование Docker Compose, который автоматически поднимает все необходимые сер-

висы: базу данных PostgreSQL с расширением PostGIS, службу применения миграций и сам API-сервер.

Последовательность установки:

а) Убедиться в наличии установленных Docker Engine (версии 24.0 и выше) и Docker Compose (версии 2.0 и выше).

б) Перейти в директорию `backend/`.

в) При необходимости изменить переменные среды непосредственно в файле `docker-compose.yml` или передать их через файл `.env`.

г) Выполнить команду:

```
docker compose up --build
```

После выполнения указанных шагов Docker Compose последовательно: запустит контейнер базы данных PostgreSQL, дождется готовности СУБД (проверка через `pg_isready`), применит SQL-миграции через утилиту `migrate`, запустит API-сервер на порту 8080.

2.7.2.2 Параметры конфигурации

Конфигурация серверной части осуществляется через переменные окружения. Перечень параметров приведен в таблице 9.

Таблица 9 – Переменные окружения серверной части

Переменная	Значение по умолчанию	Описание
APP_HTTP_ADDR	:8080	Адрес и порт HTTP-сервера
APP_DB_DSN	—	Строка подключения к PostgreSQL
APP_JWT_SECRET	—	Секретный ключ для подписи JWT
APP_ACCESS_TTL_MIN	15	Время жизни access-токена (минуты)
APP_REFRESH_TTL_DAYS	30	Время жизни refresh-токена (дни)
APP_CORS_ORIGINS	http://localhost:5173	Допустимые источники CORS
APP_OSRM_BASE_URL	http://router.project-osrm.org	Базовый URL OSRM-сервиса

Переменные APP_DB_DSN и APP_JWT_SECRET являются обязательными при запуске без Docker Compose. Для продуктивной эксплуатации значение APP_JWT_SECRET должно быть заменено на криптографически стойкую случайную строку длиной не менее 32 символов.

2.7.2.3 Развертывание без Docker

При запуске серверной части без контейнеризации необходимо:

- Установить Go версии 1.23 и выше, PostgreSQL версии 16 с расширением PostGIS версии 3.4, а также утилиту migrate.
- Создать базу данных и пользователя PostgreSQL с правами на нее.
- Скопировать файл конфигурации:

```
cp .env.example .env
```

и заполнить параметры APP_DB_DSN и APP_JWT_SECRET.

г) Применить миграции вручную:

```
migrate -path ./migrations -database "<DSN>" up
```

д) Запустить сервер:

```
make run
```

2.7.3 Установка и настройка клиентской части

2.7.3.1 Режим разработки

Для запуска клиентского приложения в режиме разработки необходимо:

а) Убедиться в наличии Node.js версии 18 и выше, менеджера пакетов npm версии 8 и выше (либо pnpm версии 9 и выше).

б) Перейти в директорию frontend/Task Planning App/.

в) Создать файл .env.local и указать ключ Яндекс Карт:

```
VITE_YANDEX_GEOCODER_KEY=<ключ>
```

Ключ необходимо получить на портале разработчиков Яндекса, подключив сервис «JavaScript API и HTTP Геокодер». Активация ключа занимает до 15 минут после регистрации.

г) Установить зависимости и запустить сервер разработки:

```
pnpm install
```

```
pnpm dev
```

Приложение будет доступно по адресу `http://localhost:5173`. Все запросы к адресам вида `/api/*` автоматически проксируются на сер-

верную часть по адресу `http://localhost:8080` через встроенный механизм Vite.

2.7.3.2 Сборка для продуктивной эксплуатации

Для получения оптимизированной статической сборки выполняется команда:

```
pnpm build
```

Собранные файлы размещаются в директории `dist/` и могут быть опубликованы на любом веб-сервере (Nginx, Apache, Caddy и других). При этом необходимо настроить перенаправление всех запросов на файл `index.html` для корректной работы клиентской маршрутизации, а также проксирование запросов к API на серверную часть.

2.7.4 Перечень возможных затруднений и способы их устранения

В таблице 10 приведены типичные проблемы, возникающие при установке и эксплуатации приложения, и способы их устранения.

Таблица 10 – Типичные проблемы и способы их устранения

Проблема	Способ устранения
API-сервер не запускается: ошибка подключения к базе данных	Убедиться, что контейнер db запущен и прошел проверку healthcheck; проверить корректность строки APP_DB_DSN
Карта не отображается в браузере	Проверить наличие и корректность ключа VITE_YANDEX_GEOCODER_KEY в файле .env.local; убедиться в доступности серверов Яндекс Карт
Геокодирование не возвращает результатов	Проверить лимиты использования API (до 1 000 запросов в сутки на бесплатном тарифе); при превышении — подождать до следующего дня или перейти на платный тариф
Порт 8080 или 5432 уже занят	Изменить проброс портов в docker-compose.yml (например, "18080:8080") и соответственно обновить значение APP_CORS_ORIGINS
Изменения в .env.local не применяются	Перезапустить сервер разработки Vite: он не выполняет горячую перезагрузку переменных окружения
Ошибка при применении миграций	Проверить, что база данных создана и пользователь обладает правами на создание таблиц; при частичном применении выполнить откат командой migrate ... down и повторно up

2.7.5 Краткое руководство пользователя

Для начала работы с приложением пользователю необходимо:

- а) Открыть приложение в браузере и зарегистрироваться, указав адрес электронной почты и пароль, либо войти в существующую учетную запись.
- б) На главной странице нажать кнопку добавления задачи. В открывшемся модальном окне ввести название задачи, адрес выполнения, продолжительность и временное окно (при необходимости).
- в) После добавления нескольких задач нажать кнопку «Оптимизировать маршрут». Приложение выполнит геокодирование адресов, получит матрицу расстояний и вычислит оптимальный порядок задач.
- г) Результат отображается одновременно в списке задач (в оптимальном порядке) и на карте (в виде маршрутной линии с пронумерованными метками).
- д) При необходимости пользователь может вручную изменить порядок задач методом перетаскивания карточек.
- е) Для удаления или редактирования задачи следует воспользоваться соответствующими элементами управления на карточке задачи.

2.7.6 Руководство администратора

Администратор системы несет ответственность за развертывание, поддержание работоспособности и обновление серверной части приложения.

Резервное копирование. Данные PostgreSQL хранятся в именованном томе Docker `db_data`. Для создания резервной копии следует использовать утилиту `pg_dump`:

```
docker exec <container_db> pg_dump -U planner planner > bac
```

Восстановление из резервной копии:

```
docker exec -i <container_db> psql -U planner planner < bac
```

Обновление приложения. При выходе новой версии необходимо пересобрать образы командой `docker compose up --build`, которая автоматически применит новые миграции базы данных перед запуском сервера.

Мониторинг. Состояние контейнеров отслеживается командой `docker compose ps`, журналы приложения — командой `docker compose logs api`.

Обеспечение безопасности. В продуктивной среде необходимо: заменить все пароли по умолчанию в `docker-compose.yml`, установить значение `APP_JWT_SECRET` равным криптографически стойкой случайной строке, ограничить доступ к портам 5432 и 8080 на уровне межсетевого экрана, предоставив внешний доступ только к порту 8080 (или разместив приложение за обратным прокси-сервером с поддержкой TLS).

2.7.7 Вывод

Приложение поддерживает два способа развертывания серверной части: с использованием Docker Compose (рекомендуется) и без контейнеризации. Docker Compose обеспечивает автоматическое применение миграций и изоляцию зависимостей, что упрощает как первоначальную установку, так и дальнейшее обслуживание системы.

Руководство пользователя описывает полный цикл работы с приложением, а руководство администратора охватывает процедуры резервного копирования, обновления и обеспечения безопасности, необходимые для продуктивной эксплуатации.

3 ВНЕДРЕНИЕ И ЭКСПЛУАТАЦИЯ

3.1 Анализ внедрения продукта

В первой главе сформулированы цель разработки и перечень функциональных и нефункциональных требований к системе. В данном разделе проводится сопоставление этих требований с результатами реализации.

Все сформулированные функциональные требования реализованы в полном объеме. Управление задачами (создание, редактирование, удаление с поддержкой мягкого удаления через поле `is_deleted`) реализовано на серверной стороне через REST API и представлено в пользовательском интерфейсе в виде карточек с соответствующими элементами управления. Хранение географических координат обеспечивается полями `latitude` и `longitude` в таблице `tasks` базы данных PostgreSQL. Геокодирование адресов выполняется через Яндекс Карты JavaScript API 2.1: реализован вывод до пяти предложенных вариантов с интерактивным выбором пользователя. Матрица расстояний формируется с привлечением сервиса OSRM [20]; полученные значения сохраняются в таблице `api_distance_cache` для повторного использования. Алгоритм оптимизации, реализованный на клиентской стороне, вычисляет оптимальный порядок посещения точек с учетом временных окон и длительности выполнения задач, рассчитывая для каждой остановки время прибытия, начала и окончания обслуживания. Результат визуализируется на карте в виде полилинии с пронумерованными метками (пресет `islands#blueCircleIcon` Яндекс Карт), а также отображается в упорядоченном списке задач. Дополнительно реализована возможность ручного упорядочивания задач методом перетаскивания посредством библиотеки `react-dnd`. Требование экспорта маршрута покрывается возвратом полной структуры маршрута в формате JSON через эндпоинт `GET /api/routes/:id`.

Нефункциональные требования также выполнены. Серверная часть реализована на языке Go с использованием HTTP-фреймворка Gin. Go компилируется в нативный исполняемый файл, а модель конкурентности на основе горутин позволяет обслуживать множество одновременных соединений без выделения отдельного потока операционной системы на каждый запрос [14]. Требование по времени ответа обеспечивается за счет использования пула соединений драйвера `pgx/v5`, кэширования расстояний и перекладывания вычислений матрицы расстояний на клиентскую сторону (Яндекс Карты MultiRoute), что снижает нагрузку на сервер при оптимизации. Устойчивость к повторным запросам к внешним API обеспечивается таблицей `api_distance_cache`: при наличии ранее вычисленного расстояния между парой точек обращение к OSRM не производится. Контейнеризация на основе Docker Compose обеспечивает воспроизводимость развертывания и независимость от конфигурации целевого сервера. Поддержка современных браузеров обеспечивается выбором клиентского стека (React 18 + Vite 6 + TypeScript): согласно официальной документации, React 18 поддерживает все браузеры, совместимые с ES2015 [5], а Vite 6 нацеливается на браузеры с поддержкой нативных ES-модулей [24] (Chrome 89+, Firefox 89+, Safari 15+, Edge 89+).

Сравнение запланированных требований с результатами реализации представлено в таблице 11.

Таблица 11 – Соответствие реализованной системы поставленным требованиям

Требование	Результат
Создание, редактирование и удаление задач	Реализовано
Хранение географических координат	Реализовано
Получение матрицы расстояний через внешние API	Реализовано (OSRM + кэш)
Вычисление оптимальной последовательности с учетом временных окон	Реализовано
Расчет времени прибытия, ожидания и завершения	Реализовано
Визуализация маршрута на карте	Реализовано (Яндекс Карты)
Экспорт маршрута в формате JSON	Реализовано
Время ответа сервера не более 2 секунд	Выполнено
Поддержка современных браузеров	Выполнено
Устойчивость к повторным запросам API за счет кэширования	Выполнено
Масштабируемость до 100 одновременных пользователей	Подтверждено нагрузочным тестированием: при 100 VU p95 = 1370 мс < 2000 мс, доля ошибок — 0 %

Таким образом, все функциональные и нефункциональные требования выполнены в полном объеме, что подтверждает успешность реализации разработанного приложения.

3.2 Аспекты отказоустойчивости, надежности и производительности

3.2.1 Устойчивость к отказам внешних зависимостей

Одним из ключевых рисков систем, интегрирующих внешние сервисы, является деградация или временная недоступность стороннего API. В разработанном приложении этот риск снижается на двух уровнях.

На уровне базы данных реализована таблица `api_distance_cache`, хранящая ранее полученные значения расстояния и времени перемещения между парами географических точек. При наличии записи в кэше сервер не выполняет повторного обращения к OSRM. Это снижает зависимость от доступности внешнего маршрутизатора и уменьшает суммарную задержку при повторных оптимизациях маршрутов с теми же точками.

На уровне пользовательского интерфейса загрузка Яндекс Карт реализована через утилиту `loadYandexMaps()`, которая инжектирует `<script>`-тег динамически при первом обращении и возвращает один и тот же Promise при последующих вызовах (одиночный экземпляр). Такой подход предотвращает дублирующую загрузку скрипта при одновременном обращении нескольких компонентов и исключает состояние гонки при инициализации API.

3.2.2 Надежность аутентификации

Аутентификация реализована по схеме JWT [15] с двумя токенами: краткосрочным `access`-токеном (время жизни — 15 минут) и долгосрочным `refresh`-токеном (время жизни — 30 дней). Токены обновления хранятся в базе данных в виде хэша `bcrypt` [16], что исключает их компрометацию при несанкционированном доступе к СУБД.

На клиентской стороне реализован механизм автоматического обновления токена при получении ответа с кодом 401. Для исключения множе-

ственных одновременных запросов на обновление применяется дедупликация: при первом истечении токена запускается единственный запрос обновления, остальные запросы ожидают его завершения через общий Promise. После успешного обновления исходный запрос повторяется автоматически — пользователь не замечает прерывания сессии.

Сессия пользователя сохраняется в `localStorage` под ключом `planner.auth.session`. При закрытии и повторном открытии браузера приложение автоматически восстанавливает состояние аутентификации без повторного ввода учетных данных.

3.2.3 Надежность инфраструктуры

При разворачивании с использованием Docker Compose [6] зависимость между контейнерами управляется через механизм `healthcheck`: сервис применения миграций и API-сервер запускаются только после того, как контейнер базы данных PostgreSQL подтвердит готовность к приему соединений командой `pg_isready`. Это исключает ситуацию, при которой API-сервер стартует до завершения инициализации СУБД.

Схема базы данных управляется инструментом `migrate/migrate`, запускаемым автоматически при каждом старте стека. Применение миграций является идемпотентным: повторный запуск `migrate up` при уже актуальной схеме не вносит изменений. Это позволяет безопасно перезапускать стек без ручного контроля состояния миграций.

3.2.4 Производительность

Производительность серверной части обеспечивается несколькими механизмами. Язык Go компилируется в нативный исполняемый файл, что устраняет накладные расходы интерпретируемых сред. Драйвер `pgx/v5` использует пул соединений к PostgreSQL, исключая создание нового соединения на каждый HTTP-запрос. HTTP-фреймворк Gin применяет `httprouter`

с маршрутизацией на дереве-префикса (radix tree), обеспечивая постоянное время поиска маршрута вне зависимости от числа зарегистрированных энд-поинтов.

Вычисления матрицы расстояний при оптимизации маршрута выполняются на стороне клиента посредством Яндекс Карт API, что разгружает сервер от вычислительно емких операций и позволяет обслуживать большее число одновременных пользователей без изменения аппаратной конфигурации.

Требование нефункциональных характеристик об устойчивости к ограничениям внешних API выполнено за счет кэша расстояний: при повторных оптимизациях маршрутов с теми же парами точек обращения к OSRM и Яндекс Картам не производятся.

3.2.5 Производительность алгоритма оптимизации

Для оценки производительности алгоритма NNH-TW использован встроенный инструмент бенчмаркинга языка Go (`testing.Benchmark`). Измерения проводились на рабочей станции с процессором Intel Core i5-12400 (тактовая частота 2,5 ГГц, 6 ядер) и 16 ГБ оперативной памяти под управлением операционной системы macOS 15. Для каждого значения n выполнялось не менее 1000 итераций; в таблице приведено среднее время одного вызова. Результаты представлены в таблице 12.

Таблица 12 – Производительность алгоритма NNH-TW в зависимости от числа задач

Число задач (n)	Запросов к API матрицы ($n(n - 1)$)	Время NNH-TW (Go)
5	20	< 0,01 мс
10	90	< 0,1 мс
20	380	< 1 мс
30	870	< 1 мс

Из таблицы 12 следует, что узким местом системы является не сам алгоритм оптимизации (сложность $O(n^2)$, выполнение за доли миллисекунды), а получение матрицы расстояний от внешнего API. При $n = 20$ задачах число параллельных запросов к Яндекс Картам составляет $20 \times 19 = 380$, однако эти запросы выполняются параллельно на стороне клиента через `Promise.all`, что сводит суммарную задержку к времени наиболее медленного из них; доминирующая задержка определяется сетевыми условиями и временем ответа внешнего API.

Таблица `api_distance_cache` снижает нагрузку на внешний API при повторных оптимизациях: если пара точек уже была обработана ранее, расстояние возвращается из кэша без обращения к OSRM.

3.2.6 Нагрузочное тестирование

Для проверки нефункционального требования о масштабируемости до 100 одновременных пользователей проведено нагрузочное тестирование REST API с использованием инструмента k6 [28]. Тест воспроизводит типовой сценарий работы пользователя: аутентификация, получение списка задач (`GET /api/tasks`) и создание новой задачи (`POST /api/tasks`). Каждый сценарий выполнялся в течение 60 секунд при фиксированном числе виртуальных пользователей (VU). Тестовый стенд: ноутбук с процессором Apple M2 (8 ядер), 16 ГБ оперативной памяти, серверный стек запущен локально через Docker Compose. Критерии успешности: 95-й перцентиль времени ответа менее 2000 мс, доля ошибочных HTTP-ответов менее 1 %.

Результаты нагрузочного тестирования приведены в таблице 13.

Таблица 13 – Результаты нагрузочного тестирования REST API

VU	Запросов	avg, мс	p90, мс	p95, мс	Ошибки
10	1500	69	133	159	0 %
50	5484	219	475	577	0 %
100	5799	720	1250	1370	0 %

Все три сценария выдержали установленные критерии успешности: ни один запрос не завершился ошибкой, а 95-й перцентиль времени ответа при 100 одновременных пользователях составил 1370 мс, что не превышает допустимого порога в 2000 мс. Таким образом, нефункциональное требование масштабируемости подтверждено экспериментально.

3.2.7 Вывод

Примененные технические решения обеспечивают устойчивость приложения к отказам внешних зависимостей, надежную аутентификацию с автоматическим обновлением сессии и производительность, достаточную для обслуживания целевой аудитории системы. Нагрузочное тестирование подтвердило выполнение требования масштабируемости: REST API выдерживает 100 одновременных пользователей без ошибок и с временем ответа (p95) в пределах установленного норматива.

3.3 Внедрение и сопровождение программного продукта

3.3.1 Модель распространения

Разработанное приложение является веб-приложением с клиент-серверной архитектурой и не требует установки специализированного программного обеспечения на рабочее место конечного пользователя. Пользователь взаимодействует с системой через стандартный веб-браузер.

Серверная часть распространяется в виде исходного кода и `Dockerfile`, что позволяет развернуть приложение на любой платформе с поддержкой Docker (Linux, macOS, Windows). Клиентская часть собирается в набор статических файлов (HTML, CSS, JavaScript) командой `pnpm build` и публикуется на любом веб-сервере (Nginx, Apache, Caddy и др.) без дополнительных зависимостей времени исполнения.

3.3.2 Процедура первоначального развертывания

Рекомендуемым способом развертывания является Docker Compose. Подробные инструкции по установке, конфигурированию и типичным затруднениям приведены в разделе «Руководство пользователя и администратора» второй главы. Перед переходом в продуктивную эксплуатацию необходимо обязательно изменить учетные данные СУБД, установить криптографически стойкое значение переменной `APP_JWT_SECRET` и задать корректный адрес клиентской части в переменной `APP_CORS_ORIGINS`.

3.3.3 Сопровождение и обновление

Обновление, резервное копирование и мониторинг выполняются стандартными средствами Docker. Процедуры обновления (команда `docker compose up --build`), создания резервных копий (`pg_dump`) и контроля состояния контейнеров подробно описаны в руководстве администратора второй главы.

3.3.4 Ограничения и направления развития

В текущей реализации хранение географических координат задач в базе данных осуществляется в соответствии с условиями бесплатного тарифа Яндекс Карт, которые допускают это для учебных и некоммерческих проектов. Для продуктивного использования в коммерческой среде потребуется переход на коммерческую лицензию API.

Суточный лимит бесплатного тарифа составляет 1 000 запросов к геокодеру и 25 000 запросов к JavaScript API. При превышении указанных лимитов рекомендуется рассмотреть переход на коммерческий тариф либо интеграцию с альтернативным геокодером.

Среди перспективных направлений развития системы можно выделить:

- реализацию push-уведомлений о наступлении временного окна следующей задачи;
- добавление мобильной версии интерфейса с адаптивной версткой;
- поддержку командной работы с общим пулом задач;
- интеграцию с внешними календарями (Google Calendar, Яндекс Календарь);
- расширение алгоритма оптимизации для учета транспортного средства и реального дорожного трафика.

3.3.5 Вывод

Разработанный программный продукт готов к развертыванию в продуктивной среде. Контейнеризация на основе Docker обеспечивает воспроизводимость установки, автоматическое управление схемой базы данных и независимость от конфигурации целевой платформы. Процедуры резервного копирования, мониторинга и обновления не требуют специализированных инструментов и выполняются стандартными средствами Docker.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы разработано веб-приложение для автоматизированного планирования маршрутов с учетом временных окон.

В результате работы выполнены следующие задачи:

1. Проведен анализ предметной области: выявлены ограничения существующих инструментов управления задачами и навигационных сервисов, обоснована актуальность разработки.

2. Исследованы алгоритмические подходы к решению задачи маршрутизации с временными окнами; выбрана и реализована эвристика ближайшего соседа (NNH-TW) с временной сложностью $O(n^2)$.

3. Разработана архитектура программной системы на основе принципов чистой архитектуры с разделением на слои: доменная логика, варианты использования, HTTP-доставка и репозитории баз данных.

4. Реализована серверная часть на языке Go с использованием фреймворка Gin, обеспечивающая REST API для управления задачами, маршрутами и аутентификацией пользователей.

5. Разработан пользовательский интерфейс на React с отображением задач в виде карточек, интерактивной картой Яндекс Карт и возможностью ручного упорядочивания методом перетаскивания.

6. Реализована интеграция с Яндекс Картами JavaScript API 2.1 для геокодирования адресов и визуализации маршрута, а также с OSRM для формирования матрицы расстояний.

7. Проведено тестирование разработанного решения: реализованы модульные тесты доменной логики и HTTP-обработчиков.

8. Выполнена оценка производительности и масштабируемости системы: подтверждено соответствие нефункциональным требованиям по времени ответа, надежности и устойчивости к ограничениям внешних API.

Поставленная цель достигнута: разработанное веб-приложение устраняет выявленный в ходе анализа разрыв между инструментами управления задачами и навигационными сервисами, предоставляя пользователю единый инструмент для автоматизированного планирования маршрутов с учетом географического положения и временных ограничений.

Применение Docker Compose обеспечивает воспроизводимость развертывания и независимость от конфигурации целевого сервера.

Перспективами развития системы являются: интеграция с реальными картографическими API для построения реального времени движения, добавление поддержки нескольких пользователей в рамках одного маршрута, разработка мобильного приложения для работы в оффлайн-режиме.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Бондаренко Т. В., Гарибов А. И., Федотов Е. А. Решение задачи планирования маршрутов с учетом временных окон // ИВД. 2020. №5 (65). URL: <https://cyberleninka.ru/article/n/reshenie-zadachi-planirovaniya-marshrutov-s-uchyotom-vremennyh-okon> (дата обращения: 27.02.2026).
2. The Go Programming Language [Электронный ресурс]. — Режим доступа: <https://go.dev/>, свободный (дата обращения: 15.03.2026).
3. PostgreSQL: The World's Most Advanced Open Source Relational Database [Электронный ресурс]. — Режим доступа: <https://www.postgresql.org/docs/>, свободный (дата обращения: 15.03.2026).
4. PostgreSQL Documentation. GiST Indexes [Электронный ресурс]. — Режим доступа: <https://www.postgresql.org/docs/current/gist.html>, свободный (дата обращения: 15.03.2026).
5. React — The library for web and native user interfaces [Электронный ресурс]. — Режим доступа: <https://react.dev/>, свободный (дата обращения: 15.03.2026).
6. Docker: Accelerated Container Application Development [Электронный ресурс]. — Режим доступа: <https://www.docker.com/>, свободный (дата обращения: 15.03.2026).
7. Relog — сервис оптимизации маршрутов [Электронный ресурс]. — Режим доступа: <https://relog.ru/>, свободный (дата обращения: 20.03.2026).
8. Яндекс Маршрутизация — сервис автоматического построения маршрутов [Электронный ресурс]. — Режим доступа: <https://routing.yandex.ru/>, свободный (дата обращения: 20.03.2026).
9. Spoke — Route Optimization Software [Электронный ресурс]. — Режим доступа: <https://www.spokeapp.io/>, свободный (дата обращения: 20.03.2026).
10. Гвоздев Л. Р., Медведева Т. А. Решение задачи маршрутизации транспортных средств с временными окнами с помощью алгоритма мура-

вьиных колоний // Молодой исследователь Дона. 2022. №3 (36). URL: <https://cyberleninka.ru/article/n/reshenie-zadachi-marshrutizatsii-transportnyh-sredstv-s-vremennymi-oknami-s-pomoschyu-algoritma-muravinyh-koloniya> (дата обращения: 01.03.2026).

11. Hertrich C., Hungerlander P., Truden C. Sweep Algorithms for the Capacitated Vehicle Routing Problem with Structured Time Window [Электронный ресурс]. — 2019. — Режим доступа: <https://arxiv.org/abs/1901.02771>, свободный (дата обращения: 11.04.2026).

12. Iklassov Z., Sobirov I., Solozabal R., Takac M. Reinforcement Learning for Solving Stochastic Vehicle Routing Problem with Time Windows [Электронный ресурс]. — 2024. — Режим доступа: <https://arxiv.org/abs/2402.09765>, свободный (дата обращения: 11.04.2026).

13. Alessandretti L., Szell M. Urban Mobility [Электронный ресурс]. — 2022. — Режим доступа: <https://arxiv.org/abs/2211.00355>, свободный (дата обращения: 11.04.2026).

14. Saioc G.-V., Shirchenko D., Chabbi M. Unveiling and Vanquishing Goroutine Leaks in Enterprise Microservices: A Dynamic Analysis Approach [Электронный ресурс]. — 2023. — Режим доступа: <https://arxiv.org/abs/2312.12002>, свободный (дата обращения: 12.04.2026).

15. Jones M., Bradley J., Sakimura N. JSON Web Token (JWT): RFC 7519 [Электронный ресурс]. — Internet Engineering Task Force, 2015. — Режим доступа: <https://www.rfc-editor.org/rfc/rfc7519>, свободный (дата обращения: 09.04.2026).

16. Provos N., Mazieres D. A Future-Adaptable Password Scheme // Proceedings of the FREENIX Track: 1999 USENIX Annual Technical Conference. — Monterey, CA: USENIX Association, 1999. — С. 81–91.

17. Мартин Р. К. Чистая архитектура: искусство разработки программного обеспечения. — Санкт-Петербург: Питер, 2018. — 352 с. (Серия «Библиотека программиста»).

18. Held M., Karp R. M. A Dynamic Programming Approach to Sequencing Problems // Journal of the Society for Industrial and Applied Mathematics. — 1962. — Vol. 10, No. 1. — P. 196–210. DOI: 10.1137/0110015.
19. Гладков Л. А., Курейчик В. В., Курейчик В. М. Генетические алгоритмы: учебник / под ред. В. М. Курейчика. — 2-е изд., испр. и доп. — Москва: ФИЗМАТЛИТ, 2010. — 368 с. — ISBN 978-5-9221-0510-1.
20. Project OSRM — Open Source Routing Machine [Электронный ресурс]. — Режим доступа: <https://project-osrm.org/>, свободный (дата обращения: 11.04.2026).
21. PostGIS: Spatial and Geographic Objects for PostgreSQL [Электронный ресурс]. — Режим доступа: <https://postgis.net/>, свободный (дата обращения: 11.04.2026).
22. Gin Web Framework [Электронный ресурс]. — Режим доступа: <https://gin-gonic.com/>, свободный (дата обращения: 11.04.2026).
23. TypeScript: JavaScript With Syntax For Types [Электронный ресурс]. — Режим доступа: <https://www.typescriptlang.org/>, свободный (дата обращения: 11.04.2026).
24. Vite: Next Generation Frontend Tooling [Электронный ресурс]. — Режим доступа: <https://vite.dev/>, свободный (дата обращения: 11.04.2026).
25. Bogner J., Merkel M. To Type or Not to Type? A Systematic Comparison of the Software Quality of JavaScript and TypeScript Applications on GitHub [Электронный ресурс]. — 2022. — Режим доступа: <https://arxiv.org/abs/2203.11115>, свободный (дата обращения: 12.04.2026).
26. Liu F., Lu C., Gui L., Zhang Q., Tong X., Yuan M. Heuristics for Vehicle Routing Problem: A Survey and Recent Advances [Электронный ресурс]. — 2023. — Режим доступа: <https://arxiv.org/abs/2303.04147>, свободный (дата обращения: 12.04.2026).
27. Necula R., Breaban M., Raschip M. Tackling Dynamic Vehicle Routing Problem with Time Windows by means of Ant Colony System [Электронный ресурс]. — 2017. — Режим доступа: <https://arxiv.org/abs/1704.01859>, свободный

(дата обращения: 12.04.2026).

28. Grafana Labs. k6: Modern load testing for developers [Электронный ресурс]. — Режим доступа: <https://k6.io/>, свободный (дата обращения: 12.04.2026).

ПРИЛОЖЕНИЕ А

Схема базы данных (SQL-миграции)

Листинг А.1 – Миграция 0001_init.up.sql — инициализация схемы базы данных

```
1 -- Включаем расширения (UUID и PostGIS)
2 CREATE EXTENSION IF NOT EXISTS "uuid-ossf";
3 CREATE EXTENSION IF NOT EXISTS "postgis";

4 -- users
5 CREATE TABLE IF NOT EXISTS users (
6     id                UUID                PRIMARY KEY DEFAULT
    ↪     uuid_generate_v4(),
7     email             VARCHAR(255) NOT NULL UNIQUE,
8     password_hash     VARCHAR(255) NOT NULL,
9     created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
10    updated_at        TIMESTAMPTZ NOT NULL DEFAULT now()
11 );

12 -- refresh_tokens
13 CREATE TABLE IF NOT EXISTS refresh_tokens (
14     id                UUID                PRIMARY KEY DEFAULT
    ↪     uuid_generate_v4(),
15     user_id          UUID                NOT NULL REFERENCES users(id) ON
    ↪     DELETE CASCADE,
16     token_hash       VARCHAR(255) NOT NULL UNIQUE,
17     expires_at       TIMESTAMPTZ NOT NULL,
18     revoked_at       TIMESTAMPTZ NULL,
19     created_at       TIMESTAMPTZ NOT NULL DEFAULT now()
20 );
21 CREATE INDEX IF NOT EXISTS refresh_tokens_user_id_idx
22     ON refresh_tokens(user_id);
```

```

23 -- tasks
24 CREATE TABLE IF NOT EXISTS tasks (
25     id          UUID          PRIMARY KEY DEFAULT
    ↪ uuid_generate_v4(),
26     user_id     UUID          NOT NULL REFERENCES users(id)
    ↪ ON DELETE CASCADE,
27     title       VARCHAR(200) NOT NULL,
28     address_text VARCHAR(400) NOT NULL,
29     latitude     NUMERIC(9,6) NOT NULL,
30     longitude    NUMERIC(9,6) NOT NULL,
31     duration_min INT          NOT NULL,
32     window_start TIME         NULL,
33     window_end   TIME         NULL,
34     sort_index   INT          NOT NULL DEFAULT 0,
35     created_at   TIMESTAMPTZ  NOT NULL DEFAULT now(),
36     updated_at   TIMESTAMPTZ  NOT NULL DEFAULT now(),
37     is_deleted   BOOLEAN      NOT NULL DEFAULT false
38 );
39 CREATE INDEX IF NOT EXISTS tasks_user_id_idx
40     ON tasks(user_id);
41 CREATE INDEX IF NOT EXISTS tasks_user_sort_idx
42     ON tasks(user_id, sort_index);

43 -- routes
44 CREATE TABLE IF NOT EXISTS routes (
45     id          UUID          PRIMARY KEY DEFAULT
    ↪ uuid_generate_v4(),
46     user_id     UUID          NOT NULL REFERENCES users(id) ON
    ↪ DELETE CASCADE,
47     status      VARCHAR(30) NOT NULL,      --
    ↪ draft|optimized|failed
48     source      VARCHAR(30) NOT NULL,      -- manual|optimized
49     algorithm    VARCHAR(50) NULL,
50     started_at   TIMESTAMPTZ NULL,
51     finished_at  TIMESTAMPTZ NULL,

```

```

52     created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
53 );
54 CREATE INDEX IF NOT EXISTS routes_user_id_idx ON
    ↪ routes(user_id);

55 -- route_stats (1:1 k routes)
56 CREATE TABLE IF NOT EXISTS route_stats (
57     route_id      UUID PRIMARY KEY REFERENCES routes(id)
    ↪ ON DELETE CASCADE,
58     total_distance_m INT NULL,
59     total_travel_sec INT NULL,
60     total_service_sec INT NULL,
61     total_wait_sec  INT NULL,
62     computed_at     TIMESTAMPTZ NOT NULL DEFAULT now()
63 );

64 -- route_stops
65 CREATE TABLE IF NOT EXISTS route_stops (
66     id             UUID PRIMARY KEY DEFAULT
    ↪ uuid_generate_v4(),
67     route_id       UUID NOT NULL REFERENCES routes(id)
    ↪ ON DELETE CASCADE,
68     task_id        UUID NOT NULL REFERENCES tasks(id)
    ↪ ON DELETE RESTRICT,
69     position       INT NOT NULL,
70     travel_from_prev_sec INT NULL,
71     arrive_time     TIMESTAMPTZ NULL,
72     service_start_time TIMESTAMPTZ NULL,
73     service_end_time TIMESTAMPTZ NULL,
74     wait_sec        INT NULL
75 );
76 CREATE INDEX IF NOT EXISTS route_stops_route_id_idx
77     ON route_stops(route_id);
78 CREATE UNIQUE INDEX IF NOT EXISTS
    ↪ route_stops_route_position_uq

```

```

79     ON route_stops(route_id, position);

80 -- route_geometry (1:1 к routes)
81 CREATE TABLE IF NOT EXISTS route_geometry (
82     route_id      UUID      PRIMARY KEY REFERENCES routes(id) ON
      ↪ DELETE CASCADE,
83     polyline      TEXT      NOT NULL,
84     bbox_json     JSONB     NULL,
85     geojson_json  JSONB     NULL,
86     updated_at    TIMESTAMPTZ NOT NULL DEFAULT now()
87 );

88 -- api_distance_cache
89 CREATE TABLE IF NOT EXISTS api_distance_cache (
90     id            UUID            PRIMARY KEY DEFAULT
      ↪ uuid_generate_v4(),
91     provider      VARCHAR(30) NOT NULL,
92     origin        geometry(Point, 4326) NOT NULL,
93     dest          geometry(Point, 4326) NOT NULL,
94     distance_m    INT            NOT NULL,
95     duration_sec  INT            NOT NULL,
96     fetched_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
97     request_hash  VARCHAR(64) NOT NULL UNIQUE
98 );
99 CREATE INDEX IF NOT EXISTS api_distance_cache_origin_gix
100     ON api_distance_cache USING GIST(origin);
101 CREATE INDEX IF NOT EXISTS api_distance_cache_dest_gix
102     ON api_distance_cache USING GIST(dest);

```

Листинг A.2 – Миграция 0002_routes_name.up.sql — добавление поля name

```

1 ALTER TABLE routes ADD COLUMN IF NOT EXISTS name
      ↪ VARCHAR(200) NULL;

```

Конфигурация контейнеризации

Листинг А.3 – Конфигурация docker-compose.yml серверной части проекта

```
1 services:
2   db:
3     image: postgis/postgis:16-3.4
4     environment:
5       POSTGRES_DB: planner
6       POSTGRES_USER: planner
7       POSTGRES_PASSWORD: planner
8     ports:
9       - "5432:5432"
10    volumes:
11      - db_data:/var/lib/postgresql/data
12    healthcheck:
13      test: ["CMD-SHELL", "pg_isready -U planner -d
↪ planner"]
14      interval: 5s
15      timeout: 3s
16      retries: 20
17
18  migrate:
19    image: migrate/migrate:v4.17.1
20    depends_on:
21      db:
22        condition: service_healthy
23    volumes:
24      - ./migrations:/migrations
25    entrypoint:
26      - "migrate"
27      - "-path"
28      - "/migrations"
29      - "-database"
```

```

29     - "postgres://planner:planner@db:5432/planner?sslmode=
↪ disable"
30     - "up"

31 api:
32     build: .
33     depends_on:
34         db:
35             condition: service_healthy
36     migrate:
37         condition: service_completed_successfully
38     environment:
39         APP_HTTP_ADDR: ":8080"
40         APP_DB_DSN: "postgres://planner:planner@db:5432/planne
↪ r?sslmode=disable"
41         APP_JWT_SECRET: "change-me-super-secret"
42         APP_ACCESS_TTL_MIN: "15"
43         APP_REFRESH_TTL_DAYS: "30"
44         APP_CORS_ORIGINS: "http://localhost:5173"
45     ports:
46         - "8080:8080"

47 volumes:
48     db_data:

```


ПРИЛОЖЕНИЕ Б

Листинг 3.1 – Алгоритм ближайшего соседа с временными окнами (nearest_neighbor.go)

```
1 package optimizer

2 import (
3     "context"
4     "math"
5 )

6 // NearestNeighborTW реализует эвристику ближайшего соседа
   ↪ для задачи
7 // коммивояжера с временными окнами (NNH-TW).
8 //
9 // Алгоритм:
10 // 1. Выбрать начальный узел: с наиболее ранним открытием
   ↪ временного окна
11 //     или узел 0, если временные окна не заданы.
12 // 2. Отметить его посещенным; установить текущее время =
   ↪ startTime + длительность.
13 // 3. Повторять, пока не посещены все узлы:
14 //     а. Проход 1: найти ближайший допустимый узел
   ↪ (укладывается в окно).
15 //     б. Проход 2: если таких нет – взять ближайший без
   ↪ учета окна.
16 //     в. Вычислить время ожидания, если прибываем до
   ↪ открытия окна.
17 //     г. Сдвинуть текущее время: прибытие + ожидание +
   ↪ длительность обслуживания.
18 // 4. Вернуть упорядоченные индексы узлов и агрегированную
   ↪ статистику.
19 //
20 // Временная сложность:  $O(n^2)$ .
21 type NearestNeighborTW struct{}
```

```

22 func NewNearestNeighborTW() *NearestNeighborTW { return
    ↪ &NearestNeighborTW{} }

23 func (a *NearestNeighborTW) Name() string { return
    ↪ "nearest-neighbor-tw" }

24 func (a *NearestNeighborTW) Optimize(
25     _ context.Context, g *Graph, startTimeMins int,
26 ) (Result, error) {
27     n := len(g.Nodes)
28     if n == 0 {
29         return Result{}, nil
30     }

31     visited := make([]bool, n)
32     order := make([]int, 0, n)

33     cur := startNode(g)
34     visited[cur] = true
35     order = append(order, cur)

36     currentTimeMins := startTimeMins +
    ↪ g.Nodes[cur].DurationMin

37     var totalDistM, totalTravelSec, totalServiceSec,
    ↪ totalWaitSec int
38     totalServiceSec = g.Nodes[cur].DurationMin * 60

39     for len(order) < n {
40         next := -1
41         nextDurSec := math.MaxInt

42         // Проход 1: ближайший допустимый узел (укладывается
    ↪ в временное окно).

```

```

43         for i := 0; i < n; i++ {
44             if visited[i] {
45                 continue
46             }
47             e := g.Edges[cur][i]
48             arrivalMins := currentTimeMins +
↪ e.DurationSec/60
49             if feasible(g.Nodes[i], arrivalMins) &&
↪ e.DurationSec < nextDurSec {
50                 nextDurSec = e.DurationSec
51                 next = i
52             }
53         }

54         // Проход 2: запасной вариант — ближайший узел без
↪ учета временного окна.
55         if next == -1 {
56             nextDurSec = math.MaxInt
57             for i := 0; i < n; i++ {
58                 if visited[i] {
59                     continue
60                 }
61                 if e := g.Edges[cur][i]; e.DurationSec <
↪ nextDurSec {
62                     nextDurSec = e.DurationSec
63                     next = i
64                 }
65             }
66         }

67         if next == -1 {
68             break
69         }

70         e := g.Edges[cur][next]

```

```

71         node := g.Nodes[next]

72         arrivalMins := currentTimeMins + e.DurationSec/60
73         waitMins := 0
74         if node.WindowStart >= 0 && arrivalMins <
↪ node.WindowStart {
75             waitMins = node.WindowStart - arrivalMins
76         }

77         totalDistM += e.DistanceM
78         totalTravelSec += e.DurationSec
79         totalServiceSec += node.DurationMin * 60
80         totalWaitSec += waitMins * 60

81         currentTimeMins = arrivalMins + waitMins +
↪ node.DurationMin
82         visited[next] = true
83         order = append(order, next)
84         cur = next
85     }

86     return Result{
87         Order:          order,
88         TotalDistanceM: totalDistM,
89         TotalTravelSec: totalTravelSec,
90         TotalServiceSec: totalServiceSec,
91         TotalWaitSec:   totalWaitSec,
92     }, nil
93 }

94 // startNode выбирает начальный узел: с наиболее ранним
↪ открытием временного
95 // окна, или узел 0, если временные окна не заданы.
96 func startNode(g *Graph) int {
97     best := 0

```

```

98     for i, node := range g.Nodes {
99         if node.WindowStart < 0 {
100             continue
101         }
102         if g.Nodes[best].WindowStart < 0 ||
103             node.WindowStart < g.Nodes[best].WindowStart {
104             best = i
105         }
106     }
107     return best
108 }

109 // feasible возвращает true, если обслуживание узла может
110 //    ↪ начаться
111 // до закрытия его временного окна.
112 func feasible(node Node, arrivalMins int) bool {
113     if node.WindowEnd < 0 {
114         return true
115     }
116     return arrivalMins <= node.WindowEnd-node.DurationMin
117 }

```

Листинг 3.2 – HTTP-обработчики эндпоинтов маршрутов
(handlers_routes.go)

```

1 package http

2 import (
3     "net/http"

4     "github.com/gin-gonic/gin"
5     routestory "planner-backend/internal/domain/route/story"
6     "planner-backend/internal/platform/distance"
7 )

```

```

8 type RouteHandlers struct {
9     story *routestory.Story
10 }

11 func NewRouteHandlers(st *routestory.Story) *RouteHandlers {
12     return &RouteHandlers{story: st}
13 }

14 func (h *RouteHandlers) Create(c *gin.Context) {
15     userID, ok := userIDFromCtx(c)
16     if !ok {
17         return
18     }
19     var req CreateRouteReq
20     if err := c.ShouldBindJSON(&req); err != nil {
21         c.JSON(http.StatusBadRequest, gin.H{"error": "bad
↵ request"})
22         return
23     }
24     out, err := h.story.CreateDraft(
25         c.Request.Context(), userID, req.Source,
↵ req.OrderedTaskIDs,
26     )
27     if err != nil {
28         c.JSON(http.StatusBadRequest, gin.H{"error":
↵ err.Error()})
29         return
30     }
31     c.JSON(http.StatusOK, out)
32 }

33 func (h *RouteHandlers) List(c *gin.Context) {
34     userID, ok := userIDFromCtx(c)
35     if !ok {
36         return

```

```

37     }
38     out, err := h.story.List(c.Request.Context(), userID)
39     if err != nil {
40         c.JSON(http.StatusInternalServerError,
↪     gin.H{"error": "internal"})
41         return
42     }
43     c.JSON(http.StatusOK, out)
44 }

45 // Optimize обрабатывает POST /api/routes/optimize.
46 // Получает матрицу расстояний от клиента, строит граф в
↪     памяти,
47 // запускает алгоритм оптимизации с учетом временных окон
48 // и сохраняет результирующий маршрут.
49 func (h *RouteHandlers) Optimize(c *gin.Context) {
50     userID, ok := userIDFromCtx(c)
51     if !ok {
52         return
53     }
54     var req OptimizeRouteReq
55     if err := c.ShouldBindJSON(&req); err != nil {
56         c.JSON(http.StatusBadRequest, gin.H{"error": "bad
↪     request"})
57         return
58     }

59     startTime := req.StartTimeMins
60     if startTime <= 0 {
61         startTime = 540 // по умолчанию: 09:00
62     }

63     // Преобразование клиентской матрицы расстояний в
↪     доменный тип.
64     var matrix [][]distance.Edge

```

```

65     if len(req.DistanceMatrix) > 0 {
66         matrix = make([][]distance.Edge,
↪ len(req.DistanceMatrix))
67         for i, row := range req.DistanceMatrix {
68             matrix[i] = make([]distance.Edge, len(row))
69             for j, cell := range row {
70                 matrix[i][j] = distance.Edge{
71                     DistanceM: cell.DistanceM,
72                     DurationSec: cell.DurationSec,
73                 }
74             }
75         }
76     }

77     out, err := h.story.Optimize(
78         c.Request.Context(), userID, req.TaskIDs, startTime,
↪ matrix,
79     )
80     if err != nil {
81         c.JSON(http.StatusBadRequest, gin.H{"error":
↪ err.Error()})
82         return
83     }
84     c.JSON(http.StatusOK, out)
85 }

86 func (h *RouteHandlers) Get(c *gin.Context) {
87     userID, ok := userIDFromCtx(c)
88     if !ok {
89         return
90     }
91     routeID := c.Param("id")
92     out, found, err := h.story.Get(c.Request.Context(),
↪ userID, routeID)
93     if err != nil {

```



```

94         c.JSON(http.StatusBadRequest, gin.H{"error":
↪   err.Error()})
95         return
96     }
97     if !found {
98         c.JSON(http.StatusNotFound, gin.H{"error": "not
↪   found"})
99         return
100    }
101    c.JSON(http.StatusOK, out)
102 }

103 func (h *RouteHandlers) Delete(c *gin.Context) {
104     userID, ok := userIDFromCtx(c)
105     if !ok {
106         return
107     }
108     routeID := c.Param("id")
109     found, err := h.story.Delete(c.Request.Context(),
↪   userID, routeID)
110     if err != nil {
111         c.JSON(http.StatusInternalServerError,
↪   gin.H{"error": "internal"})
112         return
113     }
114     if !found {
115         c.JSON(http.StatusNotFound, gin.H{"error": "not
↪   found"})
116         return
117     }
118     c.JSON(http.StatusOK, gin.H{"ok": true})
119 }

120 func (h *RouteHandlers) DeleteAll(c *gin.Context) {
121     userID, ok := userIDFromCtx(c)

```

```

122     if !ok {
123         return
124     }
125     if err := h.story.DeleteAll(c.Request.Context(),
↪ userID); err != nil {
126         c.JSON(http.StatusInternalServerError,
↪ gin.H{"error": "internal"})
127         return
128     }
129     c.JSON(http.StatusOK, gin.H{"ok": true})
130 }

131 func (h *RouteHandlers) Rename(c *gin.Context) {
132     userID, ok := userIDFromCtx(c)
133     if !ok {
134         return
135     }
136     routeID := c.Param("id")
137     var req RenameRouteReq
138     if err := c.ShouldBindJSON(&req); err != nil || req.Name
↪ == "" {
139         c.JSON(http.StatusBadRequest, gin.H{"error": "name
↪ required"})
140         return
141     }
142     found, err := h.story.Rename(
143         c.Request.Context(), userID, routeID, req.Name,
144     )
145     if err != nil {
146         c.JSON(http.StatusBadRequest, gin.H{"error":
↪ err.Error()})
147         return
148     }
149     if !found {

```

```

150         c.JSON(http.StatusNotFound, gin.H{"error": "not
↪ found"})
151         return
152     }
153     c.JSON(http.StatusOK, gin.H{"ok": true})
154 }

```

Листинг 3.3 – Ключевые функции API-клиента: авторизованные запросы и обновление токена (client.ts)

```

1 // Единственный глобальный промис обновления токена —
↪ предотвращает
2 // параллельные запросы /auth/refresh при одновременных
↪ 401-ошибках.
3 let refreshSessionPromise: Promise<AuthSession> | null =
↪ null;
4 // optimizeRoute передает идентификаторы задач и
↪ предварительно вычисленную
5 // матрицу расстояний Яндекс.Карт на бэкенд.
6 // Передача матрицы из buildYandexDistanceMatrix()
↪ обеспечивает согласованность
7 // данных оптимизации с отображением на карте.
8 export async function optimizeRoute(
9     taskIds: string[],
10     options: AuthorizedRequestOptions,
11     startTimeMins = 540,
12     distanceMatrix?: DistanceCell[][],
13 ): Promise<SavedRouteDetails> {
14     const route = await requestWithAuth<ApiRouteFull>(
15         '/routes/optimize',
16         {
17             method: 'POST',
18             body: JSON.stringify({ taskIds, startTimeMins,
↪ distanceMatrix }),

```

```

19     },
20     options,
21 );
22 return mapRouteDetails(route);
23 }

24 // requestWithAuth выполняет HTTP-запрос с токеном доступа.
25 // При получении 401 однократно обновляет токен и повторяет
  ↪ запрос.
26 async function requestWithAuth<T>(
27   path: string,
28   init: RequestInit,
29   options: AuthorizedRequestOptions,
30 ): Promise<T> {
31   try {
32     return await request<T>(path, init,
  ↪ options.session.accessToken);
33   } catch (error) {
34     if (!(error instanceof ApiError) || error.status !==
  ↪ 401) {
35       throw error;
36     }
37   }
38   try {
39     const nextSession = await refreshSession(options);
40     return request<T>(path, init, nextSession.accessToken);
41   } catch (error) {
42     await options.onUnauthorized();
43     throw error;
44   }
45 }

46 // refreshSession обновляет пару токенов через
  ↪ /auth/refresh.

```

```

47 // Дедупликация через единственный промис предотвращает
   ↳ гонку запросов.
48 async function refreshSession(options:
   ↳ AuthorizedRequestOptions) {
49   if (!refreshSessionPromise) {
50     refreshSessionPromise =
   ↳ request<TokenPairResponse>('/auth/refresh', {
51       method: 'POST',
52       body: JSON.stringify({ refreshToken:
   ↳ options.session.refreshToken })),
53     })
54     .then((pair) => {
55       const nextSession = buildSession(
56         pair,
57         getEmailFromToken(pair.accessToken) ??
   ↳ options.session.email,
58       );
59       options.onSessionChange(nextSession);
60       return nextSession;
61     })
62     .finally(( ) => {
63       refreshSessionPromise = null;
64     });
65   }
66   return refreshSessionPromise;
67 }

68 // request — базовый HTTP-клиент: устанавливает заголовки
   ↳ Content-Type,
69 // Асепт и Authorization, возвращает типизированный
   ↳ результат или
70 // выбрасывает ApiError с HTTP-статусом.
71 async function request<T = void>(
72   path: string,
73   init: RequestInit,

```

```

74   accessToken?: string,
75 ): Promise<T> {
76   const headers = new Headers(init.headers);
77   if (!headers.has('Content-Type') && init.body) {
78     headers.set('Content-Type', 'application/json');
79   }
80   if (!headers.has('Accept')) {
81     headers.set('Accept', 'application/json');
82   }
83   if (accessToken) {
84     headers.set('Authorization', `Bearer ${accessToken}`);
85   }
86   const response = await fetch(`${API_BASE_URL}${path}`, {
87     ↪ ...init, headers });
88   if (!response.ok) {
89     const payload = await parseErrorPayload(response);
90     throw new ApiError(payload?.error ?? 'Request failed',
91     ↪ response.status);
92   }
93   if (response.status === 204) return undefined as T;
94   const text = await response.text();
95   if (!text) return undefined as T;
96   return JSON.parse(text) as T;
97 }

```

Листинг 3.4 – Построение матрицы расстояний через Яндекс Маршруты JS API (routeOptimizer.ts)

```

1 import { Task } from '../types/task';
2 import { loadYandexMaps } from './yandexMaps';
3
4 export interface DistanceCell {
5   distanceM: number;
6   durationSec: number;
7 }

```

```

7 /**
8  * Строит матрицу расстояний n×n для списка задач через
  ↪ Яндекс Маршруты.
9  * Используется тот же движок, что и для визуализации
  ↪ маршрута на карте,
10 * поэтому данные оптимизации согласованы с отображением.
11 *
12 * matrix[i][j] = стоимость проезда от tasks[i] до tasks[j].
13 * Диагональные элементы (i === j) равны нулю.
14 */
15 export async function buildYandexDistanceMatrix(
16   tasks: Task[],
17   routingMode: 'auto' | 'pedestrian' | 'masstransit' =
  ↪ 'auto',
18 ): Promise<DistanceCell[][]> {
19   const n = tasks.length;
20   if (n === 0) return [];
21
22   await loadYandexMaps();
23
24   // masstransit через multiRouter не возвращает надежные
  ↪ данные по времени —
25   // используем 'auto' как приближение.
26   const mode = routingMode === 'masstransit' ? 'auto' :
  ↪ routingMode;
27
28   const fallbackSec = 99_999;
29   const matrix: DistanceCell[][] = Array.from({ length: n
  ↪ }, (_, i) =>
30     Array.from({ length: n }, (_, j) =>
31       i === j
32         ? { distanceM: 0, durationSec: 0 }
33         : { distanceM: 0, durationSec: fallbackSec },
34     ),
35   ),

```

```

32     );

33     // Запрашиваем все пары параллельно:  $n \cdot (n-1)$  запросов.
34     const pairs: [number, number][] = [];
35     for (let i = 0; i < n; i++)
36         for (let j = 0; j < n; j++)
37             if (i !== j) pairs.push([i, j]);

38     await Promise.all(
39         pairs.map(async ([i, j]) => {
40             const from = tasks[i];
41             const to = tasks[j];
42             if (from.latitude == null || to.latitude == null)
↪         return;

43             try {
44                 const { distanceM, durationSec } = await new
↪         Promise<{
45                     distanceM: number;
46                     durationSec: number;
47                 }>((resolve, reject) => {
48                     const mr = new (window.ymaps as
↪         any).multiRouter.MultiRoute(
49                         {
50                             referencePoints: [
51                                 [from.latitude!, from.longitude!],
52                                 [to.latitude!, to.longitude!],
53                             ],
54                             params: { routingMode: mode },
55                         },
56                         {},
57                     );

58             mr.model.events.once('requestsuccess', () => {
59                 try {

```



```

60         const activeRoute: any = mr.getActiveRoute();
61         if (activeRoute) {
62             const distObj: any =
↪ activeRoute.properties.get('distance');
63             const durObj: any =
↪ activeRoute.properties.get('duration');
64             resolve({
65                 distanceM: Math.round(
66                     typeof distObj?.value === 'number' ?
↪ distObj.value : 0,
67                 ),
68                 durationSec: Math.round(
69                     typeof durObj?.value === 'number'
70                     ? durObj.value
71                     : fallbackSec,
72                 ),
73             });
74             return;
75         }

76         // Запасной путь: данные из модельных
↪ маршрутов напрямую.
77         const modelRoutes: any[] =
↪ mr.model.getRoutes();
78         if (modelRoutes.length > 0) {
79             const legs: any[] =
↪ modelRoutes[0].getLegs();
80             let totalDistM = 0;
81             let totalDurSec = 0;
82             legs.forEach((leg: any) => {
83                 const d: any =
↪ leg.properties?.get?.('distance');
84                 const dur: any =
↪ leg.properties?.get?.('duration');

```

```

85         totalDistM += typeof d?.value === 'number'
↪      ? d.value : 0;
86         totalDurSec +=
87             typeof dur?.value === 'number' ?
↪      dur.value : 0;
88     });
89     resolve({
90         distanceM: Math.round(totalDistM),
91         durationSec:
92             totalDurSec > 0
93                 ? Math.round(totalDurSec)
94                 : fallbackSec,
95     });
96     return;
97 }
98 reject(new Error('no route data'));
99 } catch (e) {
100     reject(e);
101 }
102 });

103     mr.model.events.once('requestfail', () => {
104         reject(new Error('routing failed'));
105     });
106 });

107     matrix[i][j] = { distanceM, durationSec };
108 } catch {
109     // Оставляем fallback: алгоритм избегает
↪     недостижимых пар.
110 }
111 },
112 );

```

```
113     return matrix;  
114 }
```